RoboSeed: Multidimensional Evaluation of Vision–Language–Action and World–Action Models

Capability, Resource Affordability, and Research Evidence

Xin Ma[1], Furong Chen[2], Xinlong Man[2], Bo Zhang[1], Yalong Guo[3], Haolan Lei[4]
Chaochen Cui[2], Hongyuan Zhang[5], Mulin Chen[1], Yi Wang[6], Lap-pui Chau[6], Chuang Yang[6*P]

[1]Northwestern Polytechnical University
[2]Jiangsu Yunmu Intelligent Manufacturing Co., Ltd.
[3]National University of Singapore
[4]Xi'an University of Posts and Telecommunications
[5]The University of Hong Kong
[6]The Hong Kong Polytechnic University

October 2026

Project repository: https://github.com/PrimitiveX/RoboSeed

Abstract

Choosing a robot action model requires evidence about task capability, adaptation cost, deployment requirements, and the reliability of the reported conclusions. Success rates alone provide limited guidance for community researchers working with different computing budgets and research goals. We propose the **RoboSeed evaluation framework**, a multidimensional scoring system for assessing vision–language–action (VLA) and world–action models (WAM) through a common set of decision criteria. RoboSeed is a continuing program to train, evaluate, and openly release robot models for different computing budgets, accompanied by verifiable evidence for model selection. This report connects that goal to measured results and an explicit assessment protocol; Hugging Face publication of our trained checkpoints and associated experimental results is planned.

The framework defines five dimensions: **capability, downstream training affordability, deployment affordability, locally verified research value, and reproduction evidence**. It maps empirical measurements and explicit audit criteria to 0–100 profiles using fixed normalization anchors, and provides balanced, training-constrained, and deployment-constrained preference settings. Interface and protocol checks establish when a comparison is appropriate. Missing measurements remain explicit, incomplete aggregate scores are withheld, and weighted input coverage identifies the evidence still needed for a decision. The resulting profiles help researchers screen feasible models, assess the support for mechanism claims, and prioritize follow-up experiments.

We instantiate the framework on 12 checkpoint units across ABot-M0, LingBot-VLA 2.0, OpenWAM, and LiLa-WAM, supported by local training, released-weight evaluation, task-level audits, and a PX evaluation-system study. The evidence reveals budget-dependent backbone choices, substantial sensitivity to adaptation recipes, and differences between language and visual task interfaces. Reproducible scoring code and machine-readable evidence support extension to additional models. The current study provides dimension-level assessments and actionable measurement priorities; incomplete serving traces and heterogeneous protocols leave overall numerical rankings unavailable.

Keywords: multidimensional evaluation; model selection; vision-language-action models; world-action models; research evidence.

*Corresponding author. PProject Leader.
Email: omtcyang@gmail.com

Contents

1 Introduction . . . 4
1.1 From Task Success to Research Decisions . . . 4
1.2 The Proposed RoboSeed Evaluation Framework . . . 4
1.3 Contributions . . . 5
1.4 Empirical Scope and Report Organization . . . 5
2 Related Work . . . 6
3 Empirical Evaluation of Vision–Language–Action Models . . . 6
3.1 Evaluation Protocol and Evidence Collection . . . 6
3.1.1 Experimental Axes and Comparison Principles . . . 7
3.1.2 Artifacts for Reproducible Releases . . . 7
3.1.3 Metrics and Aggregation . . . 7
3.2 ABot-M0 . . . 8
3.2.1 Architecture and Backbone Substitution . . . 8
3.2.2 Action Learning Objective . . . 8
3.2.3 Experimental Setup . . . 8
3.2.4 Backbone Scale and Task Success . . . 9
3.2.5 Training Memory, Runtime, Throughput, and Weights . . . 10
3.2.6 Loss Convergence and Closed-Loop Monitoring . . . 11
3.2.7 Rollout Concurrency and Runtime . . . 12
3.2.8 Task-Level Profiles and Uneven Scale Benefits . . . 13
3.2.9 LIBERO and Zero-Shot Robustness . . . 13
3.2.10 RoboCasa: High-Dimensional Control and Failure Diagnosis . . . 14
3.2.11 CALVIN: Continuous Language-Conditioned Task Chains . . . 15
3.2.12 Interpretation and Evidence Limits . . . 17
3.3 LingBot-VLA 2.0 . . . 17
3.3.1 Mechanism and Reproduction Boundary . . . 17
3.3.2 Experimental Setup . . . 17
3.3.3 RoboTwin Success Results . . . 18
3.3.4 Training Resources and Rollout Runtime . . . 18
3.3.5 Training Convergence . . . 18
3.3.6 Task-Level Profiles and Failure Diagnosis . . . 18
4 Empirical Evaluation of World–Action Models . . . 20
4.1 OpenWAM . . . 20
4.1.1 World–Action Coupling and Interfaces . . . 20
4.1.2 Experimental Setup . . . 20
4.1.3 RoboTwin Success Results . . . 21
4.1.4 Training Resources and Rollout Runtime . . . 22
4.1.5 Training Convergence . . . 22
4.1.6 Task-Level Profiles and Failure Diagnosis . . . 22
4.1.7 Algorithmic and Practical Value . . . 22
4.1.8 Additional Video-Backbone Training Prefix . . . 24
4.2 LiLa-WAM . . . 24
4.2.1 Compact Latent Prediction and Task Specification . . . 24
4.2.2 Experimental Setup . . . 24
4.2.3 RoboTwin Success Results . . . 24
4.2.4 Local Training Convergence and Saved Prefixes . . . 25
4.2.5 Training and Deployment Resource Boundaries . . . 26
4.2.6 Task-Level Profiles and Evidence Limits . . . 26
5 The Proposed RoboSeed Evaluation Framework . . . 26
5.1 Decision Unit, Eligibility, and Interpretation . . . 26
5.2 Quantitative Scoring Rules . . . 27
5.2.1 Capability and Sampling Uncertainty . . . 27
5.2.2 Training Affordability and Lifecycle Scope . . . 27
5.2.3 Deployment Affordability and Required Measurements . . . 28
5.2.4 Locally Verified Algorithmic Value . . . 28
5.2.5 Reproduction Evidence . . . 29
5.3 Aggregation, Missing Data, and Sensitivity . . . 29
5.4 Scientific Conclusions and Practical Choices . . . 31
5.5 PX Evaluation Harness: Systems Reproduction . . . 32
5.5.1 What Is Being Compared . . . 32

5.5.2 Experimental Setup . . . 32
5.5.3 Measured Speed, Bottlenecks, and Limits . . . 32
5.5.4 Full-Task Validation and Updated Throughput Comparison . . . 33
5.5.5 Four-GPU Topology and Batch-Cap Comparison . . . 34
6 Conclusion . . . 34
7 Future Work . . . 35
A Experimental Details and Evidence Completeness . . . 35
A.1 Dependencies, Interfaces, and Evaluation Settings . . . 35
A.2 Available Evidence and Remaining Measurements . . . 36
A.3 Reproducing the Data and Figures . . . 36
B Complete RoboTwin Task Counts . . . 37
C CALVIN Configuration, Training, and Complete Task Counts . . . 37
D New Reproduction Counts and Provenance . . . 39
D.1 Per-Method Experimental Results and Training Curves . . . 39
D.2 All 50 New Task Counts . . . 40

1 Introduction

1.1 From Task Success to Research Decisions

Community researchers choosing a robot action model face several linked questions: can it solve the target tasks, can the team afford to adapt and deploy it, and does the available evidence justify investing in its proposed mechanism? VLA models and WAMs offer different combinations of semantic conditioning, predictive representations and action generation. Their practical value depends on the task interface, computing budget and strength of the experimental evidence as well as the final success rate.

A reported percentage leaves much of this decision unresolved. Two checkpoints with similar success can require different adaptation budgets, observation pipelines and serving resources. A strong released checkpoint can coexist with a fragile local training recipe. A compact policy may rely on task-specific visual conditions rather than accept arbitrary language instructions. Researchers therefore need an evaluation that connects task outcomes to resource requirements and the conclusions supported by the experiment.

Our goal is to continuously train, evaluate, and openly release robot models for different computing budgets, while providing verifiable evidence for model selection. The report, model checkpoints, experimental results, and reproduction materials are intended to form a connected resource that community researchers can inspect, use, and extend. We plan to publish our trained checkpoints and associated experimental results on Hugging Face, with explicit links to the corresponding configurations and report versions. Public checkpoint download entries remain pending in the current snapshot.

RoboSeed addresses the model-selection need by proposing a **multidimensional evaluation and scoring framework** for the research community. We organize model selection around a common set of criteria, make their scoring rules reproducible, and connect each judgment to its underlying measurements. Training and evaluation across VLA and WAM families supply the empirical basis for these assessments. This organization lets researchers use the same evaluation questions across different architectures while preserving native task interfaces and experimental conditions.

1.2 The Proposed RoboSeed Evaluation Framework

Our framework represents each candidate by five dimensions on a 0–100 scale: capability P, downstream training affordability T, deployment affordability D, locally verified research value V, and reproduction evidence R. Table 1 connects these dimensions to the decisions they support. The evaluation unit includes the checkpoint, training recipe, execution interface and evaluator, so a score remains tied to a concrete experimental setting.

Table 1. The proposed evaluation framework connects five assessment dimensions to community research decisions. Chapter 5 defines the scoring rules and reports available scores; deployment measurements remain incomplete in this snapshot.

Dimension	Evaluation basis	Research decision
Capability (P)	Clean and Randomized closed-loop task outcomes	Does the candidate meet the required task-success level?
Training affordability (T)	Measured downstream H200 GPU-hours	Can the team afford adaptation and repeated experiments?
Deployment affordability (D)	Weight size, inference peak memory and p95 latency	Can the policy fit the serving budget and response requirement?
Research value (V)	Inspected mechanism, comparators, controlled ablations and transfer evidence	Which conclusions can inform further mechanism research?
Reproduction evidence (R)	Configuration mapping, task records, coverage, training records and seeds	How well can the reported result be checked and reproduced?

Three design choices make the profiles reusable. First, fixed normalization anchors give cost scores a declared budget interpretation and keep existing scores stable as candidates are added. Second, balanced, training-constrained and deployment-constrained weight settings expose how research priorities affect aggregate preferences. Third, explicit missing-data rules retain unavailable measurements as dashes and withhold incomplete composites; weighted input coverage identifies which observations remain to be collected. Raw metrics accompany the scores so researchers can change anchors and weights for their own budgets.

The intended use is to check task and execution compatibility, enforce capability and resource require-

ments, and then inspect the feasible candidates' dimension profiles under declared preferences. The research-value and reproduction-evidence dimensions further identify useful hypotheses and gaps in experimental support. This provides a broader basis for community evaluation while keeping the interpretation of each dimension explicit. The weights and thresholds are a versioned decision policy; their usefulness across additional laboratories and applications remains to be tested.

1.3 Contributions

A multidimensional evaluation framework for community model selection. We propose five complementary scoring dimensions that bring capability, training and deployment affordability, locally established research value, and reproduction evidence into one assessment. Fixed scoring anchors, explicit audit criteria, alternative preference profiles and missing-data rules make the framework inspectable and adaptable to different research budgets. Chapter 5 specifies the complete method.

An empirical assessment across VLA and WAM families. We apply the framework to 12 checkpoint units from four policy families, distinguishing released-weight evaluation, local adaptation and unavailable measurements. Backbone comparisons, within-family recipe comparisons and task-level diagnosis connect the scores to concrete decisions: selecting an affordable configuration, identifying a recipe gap, or determining which experiment is needed before attributing a gain to a mechanism. Native conditioning, inherited pre-train and evaluator differences remain attached to every comparison.

An auditable basis for model reuse and continuing evaluation. We provide scoring code, machine-readable profiles, source manifests, training observations and task-level outcomes, alongside a systems study of evaluation throughput. These artifacts connect model-selection conclusions to the experiments that support them. Planned checkpoint releases will pair trained weights with effective configurations, normalization statistics, compatible implementation revisions and corresponding evaluation results. As models and evidence are added, versioned report updates will retain the provenance and completion status of each experimental unit, allowing researchers to reuse the models and reassess their choices.

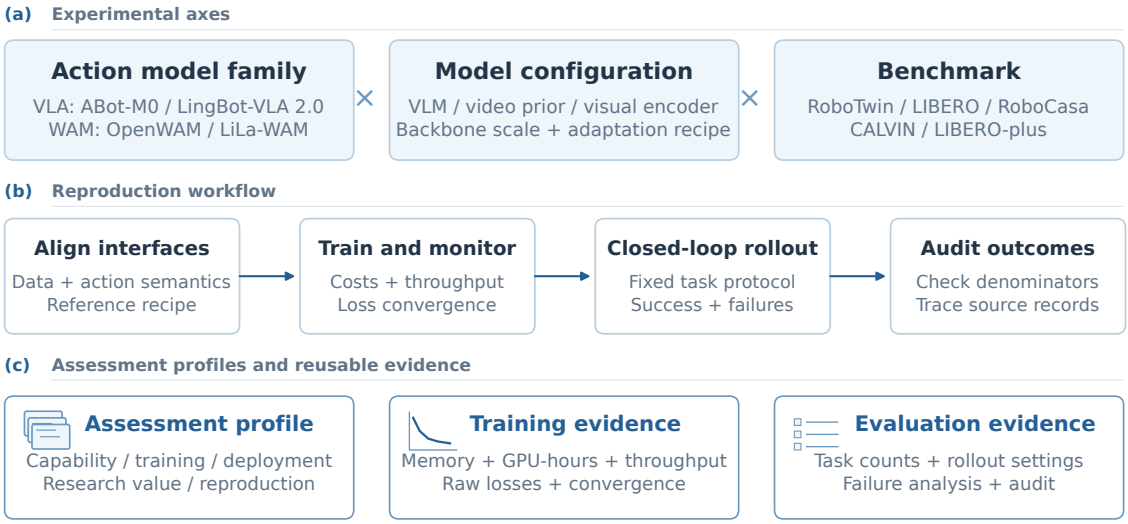


Figure 1. The RoboSeed evaluation program. (a) Four VLA/WAM families, their native model configurations and benchmarks supply the empirical assessment matrix. (b) Interface alignment, training measurements, closed-loop outcomes and evidence auditing produce the inputs to the proposed framework. (c) Five-dimensional assessment profiles are accompanied by reusable training and evaluation evidence. Chapter 5 defines scoring anchors, preference weights and missing-data rules; Chapters 3 and 4 provide the supporting model studies. The October 28 audit includes completed ABot Qwen3.5-2B and local LiLa evaluations, with the earlier evidence retained where unchanged.

1.4 Empirical Scope and Report Organization

The October 28 evidence refresh covers six complete ABot-M0 backbone-scale comparisons on RoboTwin and Qwen3.5-4B experiments on LIBERO, LIBERO-plus, RoboCasa GR1 Tabletop, and CALVIN. LIBERO, RoboCasa, and CALVIN use independently trained, dataset-specific checkpoints; LIBERO-plus evaluates the LIBERO checkpoint without additional training. The previously omitted Qwen3.5-2B evaluation is now verified over 10,000 episodes. LingBot-VLA 2.0 and OpenWAM retain their official/local results; LiLa-WAM now includes both official and local stage-2 Clean evaluation. New PX runs extend systems

evaluation to all 50 Randomized tasks. An OpenWAM Wan2.1-VACE-1.3B training prefix is documented separately without assigning a success score. Source hashes, incomplete runs and access limitations are recorded in the audit materials.

The report follows the path from empirical evidence to assessment. Chapter 3 studies VLA capability, backbone costs and adaptation sensitivity through ABot-M0 and LingBot-VLA 2.0. Chapter 4 studies predictive action models, inherited priors and task-interface requirements through OpenWAM and LiLa-WAM. Chapter 5 formalizes our proposed five-dimensional scoring framework, reports all 12 checkpoint profiles, and develops budget-specific choices and measurement priorities. Section 5.5 then examines evaluation-system efficiency. Our contributions comprise the evaluation framework, its empirical application and its auditable implementation; the studied action architectures remain attributed to their original authors. All newly audited model outcomes are simulated RoboTwin evaluations.

2 Related Work

Open and compact VLA models. OpenVLA releases a 7B VLA model and fine-tuning tools for reproducing and adapting generalist policies(Kim et al., 2024). SmolVLA emphasizes affordable training, compact models, and efficient execution(Shukor et al., 2025). These projects motivate evaluating capability together with resource requirements. Previous work also examines model scale and efficiency; RoboSeed contributes a common assessment framework that connects such capability and cost observations to explicit research-value and reproduction-evidence criteria, supported by an expanding experimental matrix.

Action modeling and spatial priors. ABot-M0 combines VLM semantic features, an action expert, and optional spatial modules, and introduces Action Manifold Learning (AML)(Yang et al., 2026). Its modular structure supports backbone substitution. Our experiments retain the main action architecture and training protocol while assessing different backbones under matched task settings.

Predictive representations and world–action models. LingBot-VLA 2.0 uses predictive visual supervision and sparse action experts(Wu et al., 2026); OpenWAM studies how dedicated world and action capacity interact(Wang et al., 2026); LiLa-WAM uses a compact latent prediction space and language-free visual task conditions(Yang et al., 2026). Their distinct inherited priors, observation interfaces and task specifications motivate evaluation beyond a single success percentage. Our local evidence is assessed independently of the original papers' broader claims.

Closed-loop evaluation and robustness. RoboTwin 2.0 provides bimanual manipulation tasks with domain randomization(Chen et al., 2025). LIBERO supports evaluation of spatial, object, goal, and long-horizon manipulation(Liu et al., 2023), while LIBERO-plus probes robustness across seven perturbation dimensions(LIBERO-plus, 2025). RoboCasa GR1 Tabletop includes arms, dexterous hands, and waist control(RoboCasa, 2025). These environments test different capabilities; their task distributions, action spaces, and evaluation denominators must remain explicit when interpreting results.

Continuous multi-task evaluation. CALVIN provides language-conditioned manipulation in a simulated tabletop environment(Mees et al., 2022). Its long-horizon evaluation measures the successful prefix of a chain of five instructions. This complements isolated task success: later instructions are attempted only after earlier instructions succeed, and the physical state is carried forward within a chain. We report this composition metric separately from the episode success rates of the other benchmarks.

3 Empirical Evaluation of Vision–Language–Action Models

3.1 Evaluation Protocol and Evidence Collection

This chapter supplies the VLA evidence for the proposed evaluation framework. ABot-M0 backbone experiments quantify capability and downstream training-cost trade-offs, while LingBot-VLA 2.0 official/local comparisons examine adaptation sensitivity. Architecture and interface inspection support the research-value audit; configurations, training records and task-level outcomes support the reproduction-evidence audit. Deployment observations are retained in their native units until they meet the measurement requirements in Chapter 5. Each model's subsection connects its mechanism, experimental settings and verified results to these evaluation questions.

The shared workflow begins with the source implementation and a documented reference recipe. It checks dataset and action semantics, reconstructs the required interfaces, runs training and closed-loop evaluation, and records both capability and resource costs. Reproduction of a published setting and deliberate changes to that setting are identified explicitly, so a backbone substitution or a run without the robotic pre-train stage is interpreted in its actual context. The current report includes four policy families and a separate systems study. A released checkpoint's successful rollout is distinguished from

reproducing its training procedure or isolating its proposed mechanism.

3.1.1 Experimental Axes and Comparison Principles

Let $\mathcal{B}$ denote VLM backbones and scales, $\mathcal{D}$ training datasets, and $\mathcal{M}$ action methods. The experimental space is

$$\mathcal{E} = \mathcal{B} \times \mathcal{D} \times \mathcal{M}. \tag{1}$$

Each experimental unit also records its training protocol, random seed, execution interface, and evaluator version. The three sets need not have equal cardinalities. Coverage expands as datasets, methods, and backbones are added, with the completion status of each unit recorded explicitly.

Backbone comparisons prioritize matching training data, update count, global batch size, optimizer, action-expert type, frozen modules, and evaluation protocol. Cross-attention and spatial-projection interfaces are adapted to each backbone's hidden width. **Equal update counts do not imply equal compute**: backbone size, interface width, and kernel implementations affect per-step costs. We therefore report task performance together with observed resource consumption rather than interpreting the study as a strictly equal-FLOPs scaling law.

3.1.2 Artifacts for Reproducible Releases

A RoboSeed release is intended to contain a complete experimental unit. Table 2 describes the proposed artifact protocol; Appendix A distinguishes available evidence from missing measurements.

Table 2. RoboSeed artifact protocol. Each release should state its evidence completeness.

Artifact	Contents	Purpose
Configuration and environment	Dataset versions, backbone sources, code commits, dependencies, seeds, and effective configuration snapshots	Reconstruct execution conditions
Training process	Updates, batch size, learning rates, frozen modules, loss curves, memory, throughput, wall time, and GPU-hours	Assess budgets and convergence
Model weights	Checkpoints, matching configurations, normalization statistics, file sizes, and checksums	Keep weights and execution interfaces consistent
Rollout process	Scenes, instructions, episodes, action chunks, step limits, concurrency, runtime, retries, and exclusion rules	Verify denominators and execution settings
Results and diagnosis	Task-level counts, failure videos, anomalies, diagnostic evidence, and untested hypotheses	Support verification and improvement

3.1.3 Metrics and Aggregation

For task t, success rate is $\mathrm{SR}_t = s_t/n_t$. The task macro-average is $T^{-1}\sum_t \mathrm{SR}_t$, and the episode micro-average is $\sum_t s_t / \sum_t n_t$. Each RoboTwin scene evaluates every task for 100 episodes, so these averages coincide within a scene. The two-scene average is $(\mathrm{SR}_{\text{Clean}} + \mathrm{SR}_{\text{Randomized}})/2$. LIBERO and RoboCasa likewise use equal episode counts per task.

LIBERO-plus evaluates each perturbation variant once, with different numbers of variants across suites and categories. Its **overall score is the micro-average across all variants**; category scores use their actual denominators. Scores from different benchmarks are not pooled. Differences between standard LIBERO and LIBERO-plus describe performance under their respective test distributions.

For CALVIN, let $\ell_i \in \{0, \ldots, 5\}$ be the successful prefix length of sequence i, and $C_k = N^{-1}\sum_i \mathbf{1}[\ell_i \geq k]$. The primary metric is

$$\bar{\ell} = \frac{1}{N}\sum_{i=1}^{N} \ell_i = \sum_{k=1}^{5} C_k. \tag{2}$$

We also report $C_1, \ldots, C_5$ and task-specific successes divided by actual attempts. Task denominators are unequal because a failed subtask terminates its chain; these conditional rates are diagnostic and are not substituted for the sequence metric.

3.2 ABot-M0

3.2.1 Architecture and Backbone Substitution

Figure 2 shows the implemented module relationships. Multi-view images and language instructions are encoded by the VLM. A VGGT stream provides geometric features, which are projected to the backbone's hidden width and fused through cross-attention. The action expert conditions on fused features, available robot state, and noisy actions to predict an action sequence. Predictions are unnormalized using dataset statistics and passed to the controller.

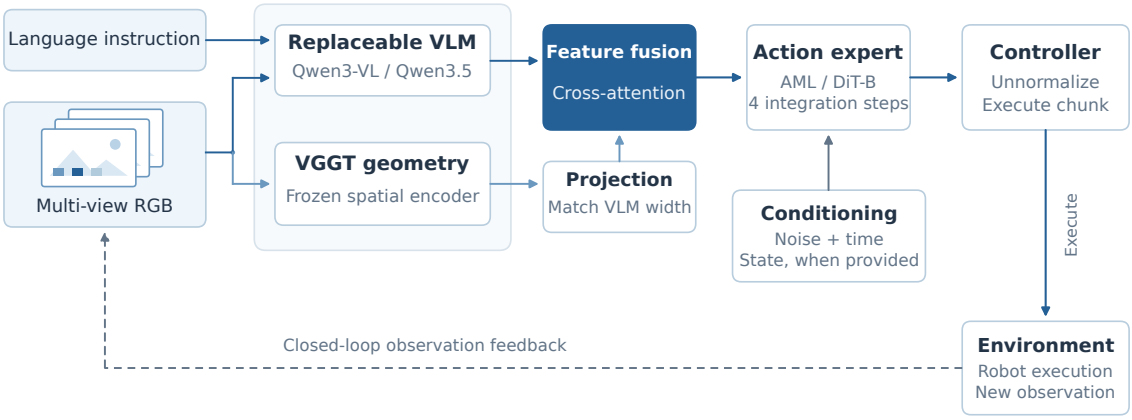


Figure 2. ABot-M0 backbone substitution and action generation, redrawn from the implementation interfaces. Feature widths are adapted when replacing the VLM; VGGT is frozen during these experiments. Dark blue arrows denote semantic and action flow, light blue arrows denote geometry flow, and the dashed connection denotes observation feedback from execution. State conditioning is used when supplied by the execution interface.

3.2.2 Action Learning Objective

For an expert action chunk A, noise ϵ, and time $\tau \in [0,1)$, the noisy input is $A^\tau = \tau A + (1-\tau)\epsilon$. The action expert predicts a clean chunk $\hat{A} = f_\theta(\phi, q, A^\tau, \tau)$, where ϕ denotes fused features and q the state condition when supplied. The implementation derives velocity fields from the predicted and target action chunks:

$$\hat{v} = \frac{\hat{A} - A^\tau}{\max(1-\tau, \delta)}, \quad v = \frac{A - A^\tau}{\max(1-\tau, \delta)}, \quad \mathcal{L} = \mathbb{E}\left[\|\hat{v} - v\|^2\right]. \tag{3}$$

Here δ ensures numerical stability. When action dimensions are padded, the loss is computed over valid dimensions. All loss curves in this report come from actual training logs. They describe optimization of the learning objective; task execution is assessed by closed-loop success.

3.2.3 Experimental Setup

Table 3. Verified experimental coverage in the current snapshot. Complete scores are derived from original evaluation artifacts.

Benchmark	Backbone	Training protocol	Evaluation coverage
RoboTwin	Qwen3.5-0.8B / 2B / 4B / 9B Qwen3-VL-2B / 4B	150k updates Final weights	50 tasks × 100 episodes per scene; 2 scenes
LIBERO	Qwen3.5-4B	40k checkpoint	4 suites; 10 tasks per suite; 50 episodes per task
LIBERO-plus	LIBERO-trained Qwen3.5-4B	Zero-shot No extra training	10,030 variants, 1 episode each
RoboCasa GR1	Qwen3.5-4B	50k checkpoint	24 tasks × 50 episodes
CALVIN	Qwen3.5-4B	34k updates Final weights	ABCD → D; 1,000 sequences of up to 5 tasks

RoboTwin training. All six evaluated configurations use four H200 NVL GPUs, global batch size 64 (16 per GPU, accumulation 1), and 150,000 updates. They initialize from the corresponding VLM and

proceed directly to the downstream post-train stage (supervised fine-tuning, SFT), without the unified robotic pre-train stage. These scores therefore describe the reproduced training setting rather than the official pre-trained release. The training data include Clean and Randomized demonstrations; experimental records report 27,500 trajectories and 6,075,103 transitions. Image resolution is 224×224, the training seed is 42, and the spatial module is frozen. Base and VLM learning rates are 10^{-5}; the action-expert rate is 10^{-4}. AdamW uses $\beta = (0.9, 0.95)$, with 5,000 warmup updates and a cosine schedule to 5×10^{-7}. The action horizon is 50; configured state and action dimensions are both 14. The action expert is DiT-B, with four repeated time samples per training example. Dependency details appear in the appendix.

RoboTwin evaluation. Every configuration evaluates 50 tasks for 100 episodes each under both `demo_clean` and `demo_randomized`, using unseen instructions and launch seed 0. The recorded evaluator commit is `13c3c47`, with the interface and frame-sampling modifications used during these runs. Scores use binary environment success counts, without mixing in progress scores. Since training includes randomized demonstrations, Randomized scores describe performance in randomized scenes; they do not establish generalization from exclusively Clean training. Comparisons with public leaderboards require matching the evaluator version and execution protocol.

Training on other datasets. LIBERO and RoboCasa are trained independently. LIBERO uses 40k updates, global batch 64, and base/VLM/action learning rates of 2.5×10^{-5}, 3×10^{-5}, and 10^{-4}. Its action horizon is 10, configured state dimension 7, and internal action representation 14-dimensional, with the active Franka channels used for execution. The saved training script specifies `include_state=false`; its effective behavior requires checking the corresponding implementation version. RoboCasa uses 24 datasets with 1,000 demonstrations each, 50k updates, and learning rates of 10^{-5}, 10^{-5}, and 10^{-4}. Its horizon is 16, state encoding uses sine and cosine of 29 joint values (58 dimensions), and actions are 29-dimensional absolute joint targets. Both experiments freeze the spatial module.

3.2.4 Backbone Scale and Task Success

Table 4. ABot-M0 backbone comparison on RoboTwin. Rates are percentages; Average equally weights the two scenes.

Backbone	Clean count	Clean	Randomized count	Randomized	Average
Qwen3.5-0.8B	4038/5000	80.76	4025/5000	80.50	80.63
Qwen3-VL-2B	4183/5000	83.66	4195/5000	83.90	83.78
Qwen3-VL-4B	4285/5000	85.70	4267/5000	85.34	85.52
Qwen3.5-2B	4215/5000	84.30	4197/5000	83.94	84.12
Qwen3.5-4B	4294/5000	85.88	4256/5000	85.12	85.50
Qwen3.5-9B	4199/5000	83.98	4184/5000	83.68	83.83

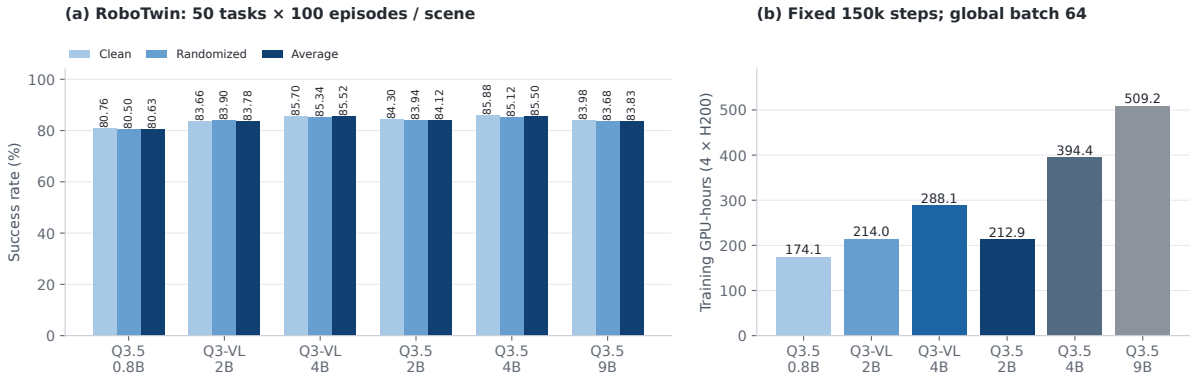


Figure 3. Task performance and training cost. Every configuration uses 150k updates and global batch 64. Bars start at zero. Training GPU-hours equal four GPUs multiplied by training wall time; they exclude evaluation and do not indicate equal-FLOPs budgets.

Smaller backbones retain substantial capability. Qwen3-VL-2B reaches 83.78%, trailing the same-family 4B score of 85.52% by 1.74 percentage points, while reducing GPU-hours from 288.1 to 214.0 (approximately 25.7%). Qwen3.5-0.8B trails its 4B counterpart by 4.87 points while reducing GPU-hours from 394.4 to 174.1 (approximately 55.8%). This evidence supports smaller backbones as useful options

for constrained budgets under the tested recipe. It does not establish that these full VLA configurations can be trained on arbitrary consumer GPUs.

The completed Qwen3.5-2B run adds a budget-relevant option. Its 100 result files match the final 100-episode counters for all 50 tasks in both scenes: 4,215/5,000 Clean (84.30%) and 4,197/5,000 Randomized (83.94%), averaging 84.12%. Recorded training is 212.9 H200 GPU-hours. This point estimate exceeds Qwen3-VL-2B by 0.34 percentage points at a very similar training cost; the single-seed experiment does not establish a robust superiority claim.

The two 4B backbones have similar aggregate scores. Qwen3-VL-4B and Qwen3.5-4B average 85.52% and 85.50%. Relative to Qwen3-VL, Qwen3.5 changes Clean success by $+0.18$ percentage points and Randomized success by -0.22 points. The experiment provides no clear aggregate success-rate advantage for the newer backbone. A single training seed does not establish statistical equivalence, and differences in architecture or pre-train recipes between families cannot be attributed to parameter count alone.

Scaling beyond 4B does not improve this recipe. The completed Qwen3.5-9B evaluation yields 4,199/5,000 Clean and 4,184/5,000 Randomized successes, for a two-scene average of 83.83%. This is 1.67 percentage points below Qwen3.5-4B while consuming 509.2 rather than 394.4 training GPU-hours, an increase of approximately 29.1%. Its average is only 0.05 points above Qwen3-VL-2B, despite approximately 2.38 times the training-device time. Under the tested recipe, expanding from smaller backbones to 4B improves the observed averages, but expanding further to 9B does not. With one training seed per configuration, these observations do not establish a universal optimum or identify the cause of the 9B result.

A second 2B family is being evaluated. Qwen3.5-2B has completed 150k updates and saved final weights. It requires 53.23 hours on four H200 GPUs (212.9 GPU-hours), processes approximately 50.1 nominal samples/s, and saves an 8.33 GB checkpoint. Its final logged action loss is approximately 1.44×10^{-3}. RoboTwin evaluation is still in progress at the snapshot cutoff; training completion and low loss are insufficient to assign a task-success score or rank it against Qwen3-VL-2B.

3.2.5 Training Memory, Runtime, Throughput, and Weights

Table 5. RoboTwin training process. Approximate memory readings retain the original records' GiB labels; uniformly sampled memory traces are unavailable. Throughput is recomputed from updates, batch size, and complete wall time.

				Memory / GPU (GiB)		
Backbone	Hours	GPU-h	Samples/s	Mean	Recorded max.	Weights (GB)
Qwen3.5-0.8B	43.53	174.1	61.3	∼52.27	∼55.06	5.04
Qwen3-VL-2B	53.51	214.0	49.8	∼54.90	∼55.30	7.77
Qwen3-VL-4B	72.03	288.1	37.0	∼83.26	∼83.93	12.58
Qwen3.5-2B	53.23	212.9	50.1	∼58.95	∼71.01	8.33
Qwen3.5-4B	98.59	394.4	27.0	∼142.50	∼143.00	13.28
Qwen3.5-9B	127.30	509.2	20.9	∼137.04	∼139.69	21.88

Training durations for the six evaluated backbones are 43:31:53, 53:30:44, 72:01:59, 53:13:34, 98:35:29, and 127:18:06 in table order. Qwen3.5-2B memory records are now available: approximately 58.95 GiB mean and 71.01 GiB maximum under the source monitoring convention. Throughput is the nominal number of batch samples processed per second, not unique data items. Checkpoint sizes come from actual file byte counts and use decimal GB (10^9 bytes). Saved weights include action, spatial, and other modules and cannot be inferred from the nominal VLM size. GPU-hours measure allocated training-device time, rather than energy, utilization, or monetary cost.

Engineering paths matter when comparing costs. At 4B, Qwen3.5 takes approximately 36.9% more training wall time and has approximately 26.9% lower nominal throughput than Qwen3-VL. The troubleshooting records identify unavailable linear-attention acceleration components and a slower PyTorch fallback path. These measurements describe the software environment used in this experiment. The benefits of enabling the accelerated implementation require a new measurement.

Memory evidence combines approximate monitoring readings with the units stated in the original records. In particular, the 142–143 readings need reconciliation with device capacity, monitoring tools, and definitions of allocated and reserved memory before formal release. We preserve these records transparently rather than treating them as exact minimum hardware requirements obtained with a uniform profiler.

The 9B run records approximately 137.04 GiB mean and 139.69 GiB peak memory, a 21.88 GB checkpoint, and 20.9 nominal samples/s. Its recorded memory is below the 4B Qwen3.5 reading despite its larger

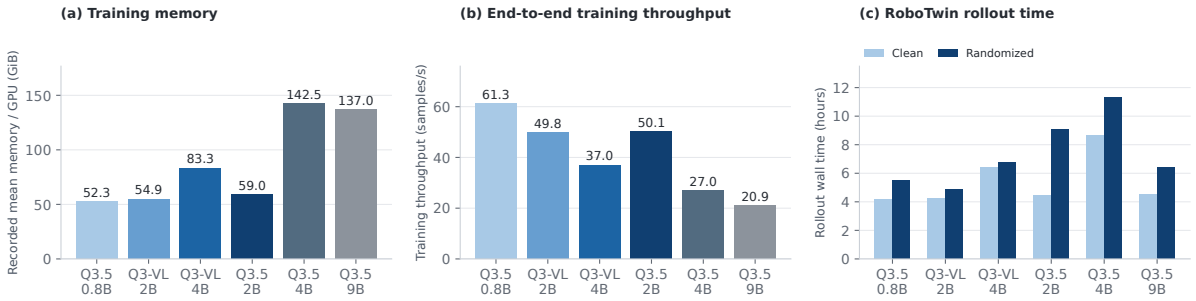


Figure 4. Training resources and rollout details. Memory bars use approximate recorded readings. The high Qwen3.5-4B reading is associated with its runtime implementation. Evaluation times retain each run's actual concurrency settings and are not measurements of single-policy inference latency.

backbone. Experiment notes report acceleration dependencies installed for later runs, but no matched before/after 4B experiment isolates their effect. Consequently, memory ordering should not be interpreted as a parameter-scaling rule or a measured kernel speedup.

3.2.6 Loss Convergence and Closed-Loop Monitoring

Figure 5 compares the six backbones through four complementary views: full training trajectories, a shared late-training zoom, last-window statistics, and the relationship between training loss and closed-loop success. Overlaying the curves exposes their similar overall shapes, while the late-stage and outcome views make their differences interpretable.

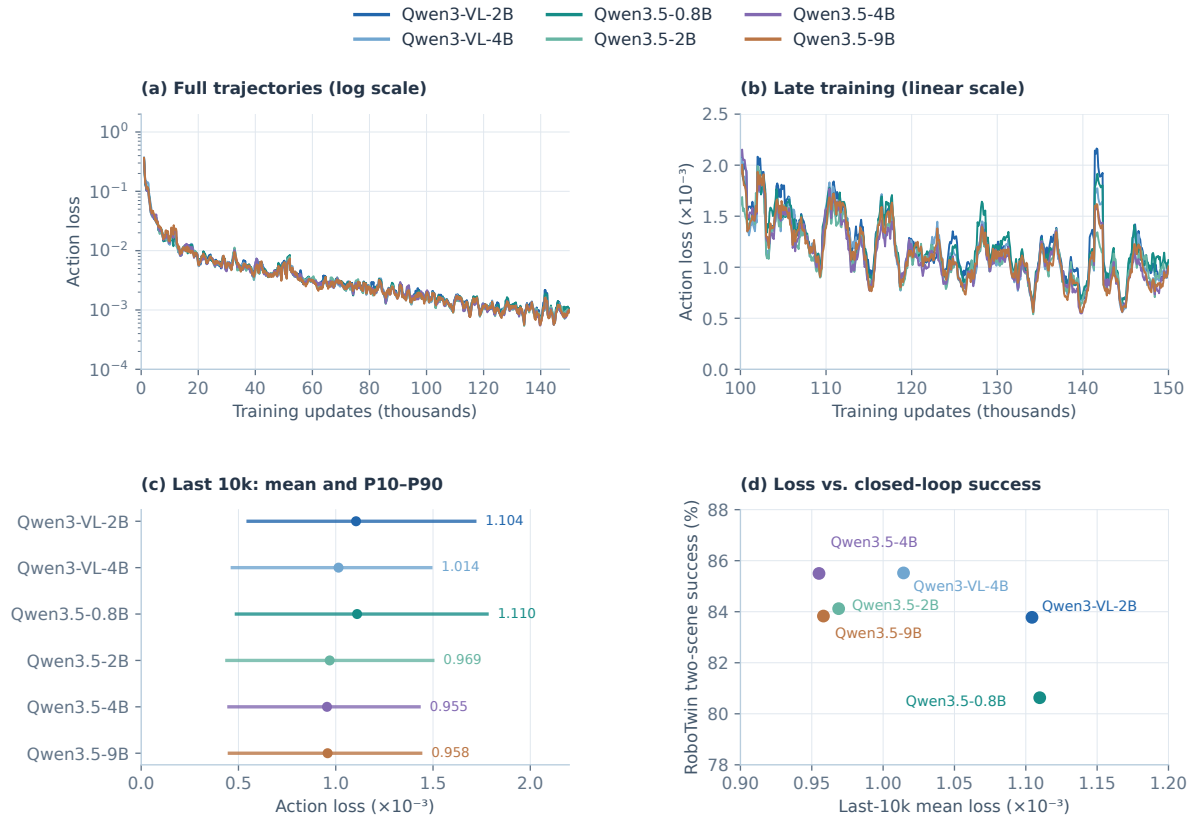


Figure 5. RoboTwin loss comparison and closed-loop outcomes. (a) All six trajectories use ten-observation moving averages on a common logarithmic scale over 0–150k updates. (b) The same smoothed series are enlarged over 100k–150k updates on a shared linear scale. (c) Dots show the mean of 100 raw observations with 140,000 $<$ step $\leq$ 150,000; horizontal lines span their empirical 10th–90th percentiles. These ranges describe within-run variation, not confidence intervals or variation across training seeds. (d) Tail means are paired with independently measured, equally weighted Clean/Randomized success rates for all six evaluated backbones. Lower training loss alone does not establish better task performance.

Similar trajectory shapes can hide modest differences in final loss. The full-run overlays share a rapid

initial decline and continued late improvement. The last-10k means range from 0.955×10^{-3} to 1.110×10^{-3}, so the smaller differences are difficult to see on a full-range logarithmic plot. The shared linear zoom exposes the late-stage levels and fluctuations without assigning each model a different vertical scale. Relative to the raw-observation mean over 100k–110k updates, last-10k means are about 24.7%–32.6% lower across the six runs, indicating continuing optimization rather than a fully flat tail.

Within-run spread accompanies each mean. The 10th–90th percentile ranges in panel (c) overlap substantially. They describe variation among logged training observations, which can reflect minibatch composition and stochastic training effects; they do not establish statistical equivalence or quantify variation across independently trained checkpoints. The raw observations remain available in the delivered CSV, and the derived means and percentiles are recorded in `robotwin_loss_statistics.csv`.

Similar tail losses do not imply similar task success. Qwen3.5-4B and Qwen3.5-9B have nearly identical last-10k mean losses (0.955×10^{-3} and 0.958×10^{-3}), yet their RoboTwin success rates are 85.50% and 83.83%. Conversely, Qwen3.5-0.8B and Qwen3-VL-2B have similar tail means near 1.11×10^{-3} but different success rates, 80.63% and 83.78%. Panel (d) connects the optimization records to the task outcomes used for model selection; it does not fit a causal or statistically validated predictive relationship.

Intermediate-checkpoint rollouts complement loss monitoring. For Qwen3.5-4B, experimental records report Randomized success of 81.4% at 110k and 85.8% at 130k, using ten episodes per task. These small evaluations screen for capability changes during training. The final checkpoint reaches 85.12% with 100 episodes per task. Differences between these evaluation sizes do not establish late-training degradation, particularly when individual ten-episode estimates are sensitive to sampling.

The 9B model finishes with action loss approximately 1.47×10^{-3}, while Qwen3.5-2B finishes near 1.44×10^{-3}. These final single observations differ from the last-10k means in panels (c,d). Qwen3.5-2B is now paired with its verified 84.12% two-scene success rate in panel (d). Loss magnitudes from different datasets or methods are not pooled, since objectives, action representations, valid channels, and task distributions differ.

3.2.7 Rollout Concurrency and Runtime

Table 6. Complete RoboTwin rollout details. Every configuration uses four evaluation GPUs. Clean and Randomized times are workflow wall times and may overlap.

Backbone	Max. workers/GPU	Clean (h)	Randomized (h)	Episodes
Qwen3.5-0.8B	6	4.15	5.52	10,000
Qwen3-VL-2B	6	4.26	4.88	10,000
Qwen3-VL-4B	4	6.38	6.77	10,000
Qwen3.5-2B	6	4.42	9.10	10,000
Qwen3.5-4B	4	8.68	11.31	10,000
Qwen3.5-9B	4	4.51	6.38	10,000

Evaluation separates policy serving from simulation and uses queues to limit concurrency. The evaluated 0.8B and 2B configurations allow up to six workers per GPU, while both 4B configurations and 9B allow four. Runtime therefore reflects backbone cost, scheduling, simulation, CPU contention, and I/O together. RoboTwin evaluation videos were disabled to reduce rendering and storage overhead, so a complete video archive is unavailable for these runs. LIBERO, LIBERO-plus, and RoboCasa retain video outputs used for aggregation or diagnosis.

Publishing these conditions helps readers distinguish workflow completion time from per-request speed. Future measurements will include policy-latency distributions, environment-step throughput, and execution rates under fixed hardware, inputs, and concurrency.

The 9B Clean and Randomized workflows take 4.51 and 6.38 hours, respectively. Their shorter elapsed times than the Qwen3.5-4B workflows do not establish faster 9B inference: the runs occurred under different software and shared-host conditions. A fixed-concurrency latency experiment is required to make that comparison.

Qwen3.5-2B Clean evaluation is recorded as 4:25:21. Its Randomized time of 9:05:43 sums the original queue and two repair queues (4:07:32, 2:47:13 and 2:10:58) following VGGT access failures affecting 19 tasks. Final scores use one complete result per task, rather than pooling failed attempts. This timing includes the repair burden and is not a matched estimate of successful-rollout speed.

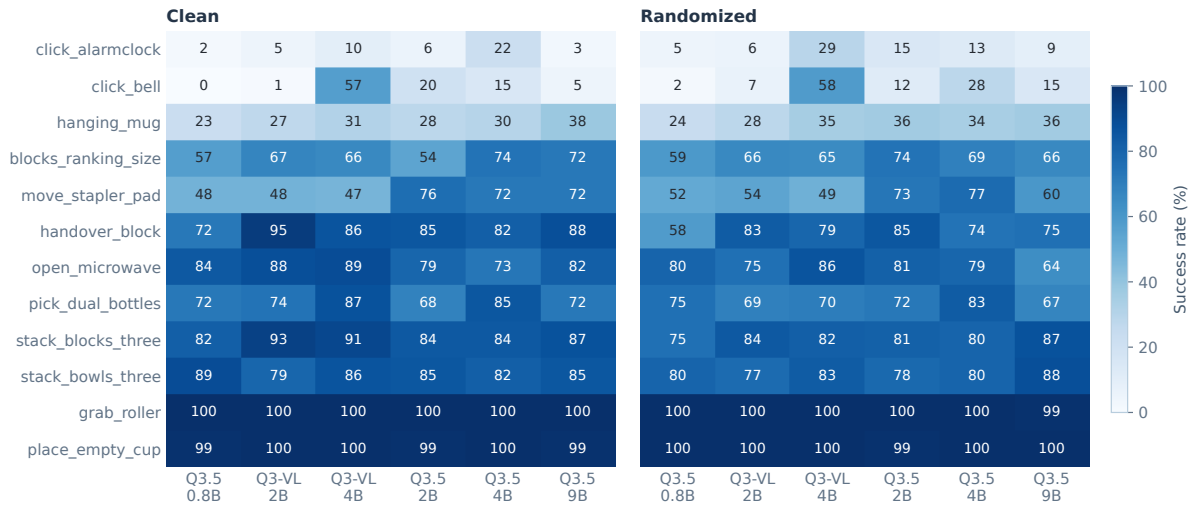


Figure 6. Success rates for selected tasks. The columns identify six evaluated configurations, including both 2B and both 4B backbones. Values are percentages. Precise contact, hanging, handover, and stacking tasks exhibit different responses to backbone scale. All 50 task counts appear in Appendix B.

3.2.8 Task-Level Profiles and Uneven Scale Benefits

Scale benefits vary by task. Clean success on `click_bell` increases from 0%/1% for the two small configurations to 57% for Qwen3-VL-4B, while Qwen3.5-4B reaches 15%. This motivates examining interactions between backbone family and task rather than relying solely on averages. Tasks such as `grab_roller` are nearly saturated across scales, whereas `hanging_mug` remains weak throughout.

Despite similar aggregate 4B scores, `move_stapler_pad`, `open_microwave`, and `click_bell` show substantial differences. These observations motivate diagnosis of data coverage, visual localization, and contact timing; they do not establish a particular module as the cause. Reported anomalies in action-dimension correlations are likewise diagnostic leads rather than proven controller-interface errors.

The 9B results further demonstrate uneven scale benefits. Compared with Qwen3.5-4B, `hanging_mug` improves from 30/34 to 38/36 Clean/Randomized successes and `stack_bowls_three` improves from 82/80 to 85/88. In contrast, `click_alarmclock` falls from 22/13 to 3/9 and `pick_dual_bottles` falls from 85/83 to 72/67. Larger capacity does not consistently remove the contact and manipulation bottlenecks in this recipe. Action-expert capacity, selected VLM feature layers, and data scale are plausible ablation axes, rather than established explanations.

3.2.9 LIBERO and Zero-Shot Robustness

Table 7. ABot-M0 with Qwen3.5-4B on single-arm tasks and perturbations. LIBERO-plus uses the same 40k LIBERO checkpoint without additional training; denominators differ by suite.

	LIBERO		LIBERO-plus	
Suite	Success / total	Rate (%)	Success / total	Rate (%)
Spatial	487/500	97.40	2072/2402	86.26
Object	500/500	100.00	2100/2518	83.40
Goal	488/500	97.60	1953/2591	75.38
Long	460/500	92.00	1902/2519	75.51
Overall micro-average	1935/2000	**96.75**	8027/10030	**80.03**

Standard LIBERO success is 96.75%, including 92.00% on the Long suite. Without additional training, the micro-average across all 10,030 LIBERO-plus variants is 80.03%. This exposes perturbation sensitivity beyond performance on the standard tasks.

Robot initial states and camera viewpoints yield 60.77% and 65.67%, below lighting (94.92%) and background textures (94.05%). The gaps motivate improvements to geometric reasoning and state conditioning. However, categories differ in perturbation difficulty and composition; the comparison does not establish causality or demonstrate understanding of all language variations.

Deduplication and result transparency. Original outputs contain duplicates for 101 tasks, including

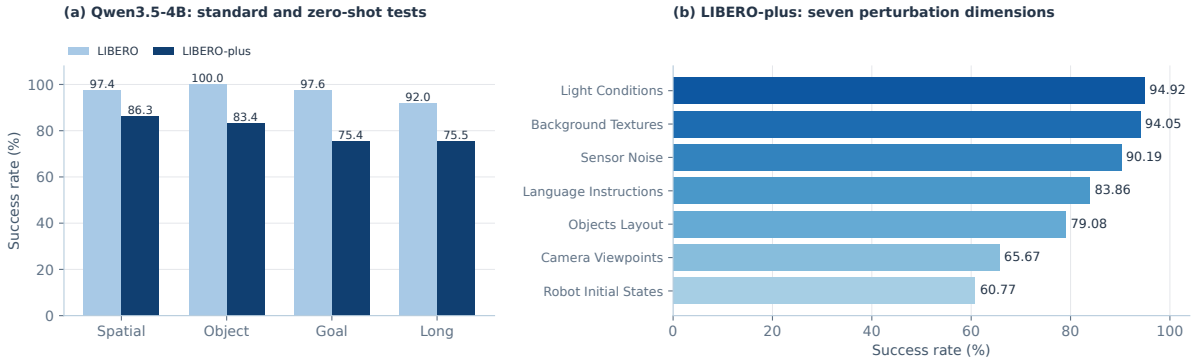


Figure 7. Performance by suite on LIBERO and LIBERO-plus, and by perturbation category on LIBERO-plus. LIBERO uses 50 episodes per task; LIBERO-plus uses one per variant. The two benchmarks are not pooled.

Table 8. LIBERO-plus perturbation categories, reaggregated from variant-level outcomes and task classifications.

Category	Success / total	Rate (%)
Background Textures	1012/1076	94.05
Camera Viewpoints	1050/1599	65.67
Language Instructions	1289/1537	83.86
Light Conditions	1084/1142	94.92
Objects Layout	1206/1525	79.08
Robot Initial States	942/1550	60.77
Sensor Noise	1444/1601	90.19

three tasks with conflicting success labels. We use (suite, task id) as the unique key and retain the output with the latest modification time as the rerun record, obtaining 8,027 successes. Assigning all three conflicting tasks either failure or success bounds the overall score at 80.01–80.04%. The selected records and all alternatives are preserved in the delivered audit data. The rerun-selection rule should be checked against run logs before formal publication. With only one episode per variant, this evaluation does not measure repeat-execution stability for individual variants.

3.2.10 RoboCasa: High-Dimensional Control and Failure Diagnosis

RoboCasa GR1 Tabletop uses 29-dimensional absolute joint targets: seven dimensions per arm, six per hand, and three for the waist. Evaluation covers 24 tasks with 50 episodes each, totaling 1,200 rollouts. Each episode allows up to 720 environment steps, and each prediction executes 12 action steps. Records report approximately 20 hours 44 minutes for 50k training updates on four H200 GPUs, averaging approximately 1.405 seconds per update. This timing is an approximate recorded statistic, unlike the exact RoboTwin wall-time values.

The policy succeeds in 574/1200 episodes (47.83%). Task scores range from 16% to 76%, with lower success on tiered shelves and some placement-and-closure tasks. Transferring an object from a cutting board to a pan reaches 76%.

Process evidence supports targeted diagnosis. Existing failure videos and probe records show rollouts that fail to approach an object before grasping, sweep it away, or continue until timeout after an unsuccessful grasp. In one tracked sequence, the end effector remains approximately 0.2–0.3 m from the target and the object barely moves, while commanded and measured joint angles differ by approximately 0.01–0.06 rad. For these observed trajectories, the evidence motivates checking the generated policy actions before attributing failure to controller tracking.

The experimenters checked action ordering, normalization statistics, absolute-joint semantics, and state encoding. An early loss plateau and outputs close to current state are consistent with a shortcut that exploits temporal continuity. **The inertial-shortcut and low-learning-rate explanations remain hypotheses**: probes do not cover every failure, and no controlled learning-rate ablation establishes causality. Follow-up experiments should vary learning rates and state conditions, measure dependence on visual input, and add periodic closed-loop monitoring.

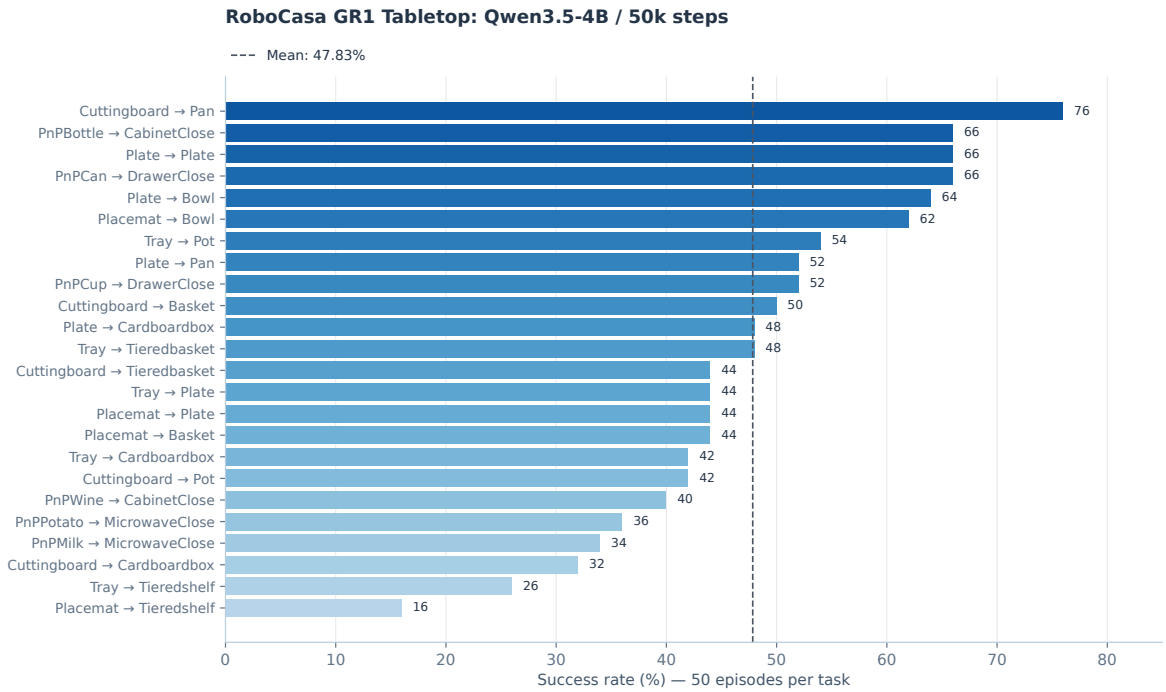


Figure 8. All 24 RoboCasa GR1 Tabletop tasks. The dashed line marks mean success of 47.83%. Equal counts of 50 episodes per task make macro- and micro-averages identical. Figure labels are abbreviated; full task names are retained in the data tables.

3.2.11 CALVIN: Continuous Language-Conditioned Task Chains

Training and interfaces. ABot-M0 with Qwen3.5-4B is trained independently on demonstrations from CALVIN environments A/B/C/D and evaluated in environment D. The converted LeRobot v2.1 dataset contains 17,870 episodes and 1,071,743 frames, with a 200×200 static camera and an 84×84 wrist camera. Images are resized for the policy interface. This ABCD $\rightarrow$ D setting includes D in training; it does not test transfer to a held-out environment D.

Training uses four H200 GPUs, global batch 64, 34,000 updates, seed 42, frozen VGGT, and a DiT-B action expert. Base/VLM/action learning rates are 2.5×10^{-5}, 10^{-5}, and 10^{-4}, with 5,000 warmup updates and a cosine schedule. The action horizon is eight. CALVIN's seven action channels (relative end-effector translation, rotation, and gripper) occupy the right-arm half of the 14-dimensional internal representation; the configured state layout is 16-dimensional after padding. The saved training script requests `include_state=false`, and the inspected rollout client sends camera images and language without an explicit robot-state field. Configured state dimensions therefore do not establish active proprioceptive conditioning.

The final run completes in 14:39:51 (14.66 hours; 58.7 GPU-hours), with approximately 41.2 nominal samples/s and a 13.27 GB final checkpoint. Its last logged action loss is 0.02949. The console file also contains preliminary runs; only the final uninterrupted 340 observations are used for its loss curve in Appendix C. Preliminary observations and the selection rule are preserved in the audit.

Evaluation protocol. Evaluation uses the stored list of 1,000 five-task sequences and CALVIN's task oracle. The environment resets to the prescribed initial condition at the start of a sequence and carries physical state between successful subtasks; the first failure terminates the chain. Each subtask has a 360-step limit. A single simulation client communicates with a separate policy server. In the inspected client, the executed chunk length is read from the checkpoint configuration and equals eight; the wrapper's `replan_steps=5` field does not control this interval. Policy buffers reset between subtasks. Evaluation wall time is recorded as approximately 10 hours 16 minutes, rather than a fixed-hardware latency measurement.

The model completes at least one task in 88.9% of sequences and all five in 57.1%. The five prefix counts sum to 3,625 successes, giving $3,625/1,000 = 3.625$. The task table independently sums to the same 3,625 successes across 4,054 reached subtask attempts. The latter denominator includes the 429 failures that terminate incomplete chains, rather than all 5,000 scheduled task positions. Sequence-level composition and conditional task success must therefore remain distinct.

Horizontal pushing is the main observed bottleneck. Six left/right block-pushing tasks account for

Table 9. CALVIN ABCD → D: successful prefixes for the final 34k Qwen3.5-4B checkpoint. Every row uses the same 1,000 sequences.

Consecutive tasks completed	Sequence count	Rate (%)
≥ 1	889/1000	88.9
≥ 2	798/1000	79.8
≥ 3	721/1000	72.1
≥ 4	646/1000	64.6
≥ 5	571/1000	57.1
Average successful sequence length (maximum 5)		**3.625**

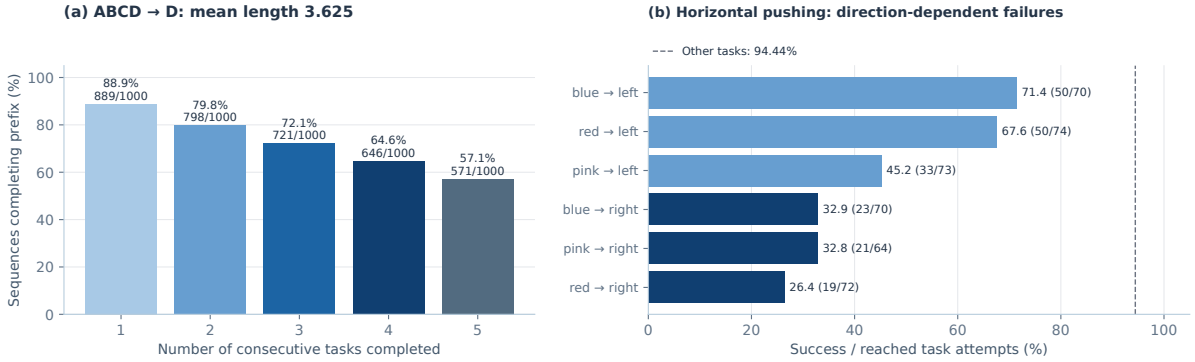


Figure 9. CALVIN chain completion and horizontal pushing. The left panel uses all 1,000 sequences as its denominator. The right panel uses actual reached attempts for each push task; the dashed reference aggregates the other 28 task types. Task frequencies and positions differ, so the reference is diagnostic rather than a matched causal comparison.

196/423 successes (46.34%), while the other 28 task types account for 3,429/3,631 (94.44%). Rightward pushes succeed in 63/206 attempts (30.58%), compared with 133/217 (61.29%) for leftward pushes. In total, 26 of the 34 task types reach at least 85% conditional success. `push_into_drawer` is retained among the other tasks because the horizontal block-pushing group is explicitly defined by the six color/direction tasks.

Table 10. CALVIN failure groups. Push left and Push right partition Horizontal push; Other tasks form its complement. Rates are conditional on reaching a task.

Task group	Success / attempts	Rate (%)
Horizontal push	196/423	46.34
Push left	133/217	61.29
Push right	63/206	30.58
Other tasks	3429/3631	94.44

The direction difference motivates checking demonstration counts, signed action distributions, initial object positions, and normalization statistics, followed by failure replay and action-horizon ablations. Contact loss during an eight-step open-loop chunk is another candidate explanation. No controlled experiment yet isolates these effects, and the aggregate rates do not justify a numerical prediction of improvement after fixing pushing.

Result selection and evidence quality. The source JSON contains two occurrences of the same top-level key: an earlier ten-sequence smoke test and the final complete evaluation. We select the final occurrence and preserve both records in `calvin_audit.json`; the smoke test is not pooled. The narrative source table omits `close_drawer`, repeats `lift_pink_block_drawer`, and overstates the number of tasks above 85%. The delivered table follows all 34 JSON task keys. Chain and task totals are internally consistent, but a per-sequence outcome file and a complete failure-video archive are unavailable in the inspected artifacts. The stored sequence list and evaluation code are hashed for provenance; external score comparisons should additionally match evaluator versions and execution settings.

3.2.12 Interpretation and Evidence Limits

The results support discussions of scale trade-offs under the specified ABot-M0 recipe and simulation settings. The following limits govern their interpretation:

- All six RoboTwin configurations have complete two-scene scores, including the newly audited Qwen3.5-2B result.
- Each completed configuration uses one training seed. One hundred episodes per task characterize that checkpoint, not variation across training runs.
- Model scales refer to nominal VLM sizes. Full policies also contain spatial modules, action experts, and projections whose sizes may change with hidden width.
- Update count, batch, and data are controlled, while backbone pre-train histories, dependency versions, and kernel paths remain systematic differences.
- Memory lacks a uniform measurement protocol, and rollout time depends on concurrency and simulation load. Neither establishes minimum deployment memory or a strict latency advantage.
- Action spaces, training steps, and learning rates differ across datasets. The cross-dataset scores do not form a single scaling law. This report includes no real-robot results.
- CALVIN measures successful prefixes of continuous task chains. Its ABCD training includes D; conditional task rates and sequence length do not constitute a held-out-environment transfer score. Available outcomes are aggregate records rather than independently audited per-sequence traces.

These evidence gaps should accompany performance and weights so readers can identify what supports each conclusion and what measurements remain necessary.

3.3 LingBot-VLA 2.0

3.3.1 Mechanism and Reproduction Boundary

LingBot-VLA 2.0 combines a Qwen3-VL-4B prefix with sparse action experts and predictive visual supervision(Wu et al., 2026). The inspected policy uses 32 routed experts, top-4 routing and a shared expert in the action stream. Depth and video teachers supervise current and future queries. These mechanisms target control specialization and predictive representations; their benefits depend on the optimization and data regime. A sparse action expert does not make the dense VLM, camera encoding, communication, or teacher supervision free.

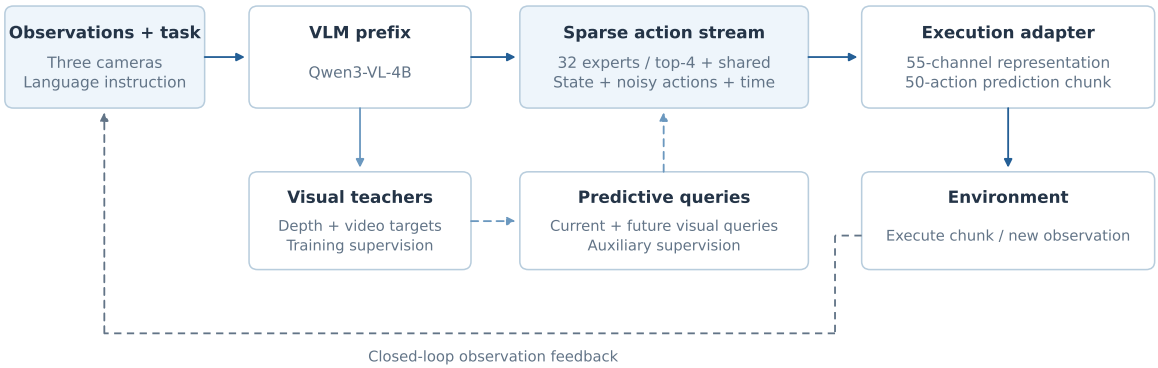


Figure 10. LingBot-VLA 2.0 reproduction interfaces. Multi-view images and language enter Qwen3-VL-4B. The action stream uses 32 routed experts with top-4 routing and a shared expert. Depth and video teachers supervise predictive queries during training; dashed arrows denote training supervision or environment feedback.

3.3.2 Experimental Setup

The local RoboTwin recipe uses Muon, three training GPUs, micro-batch 32, global batch 96, 50k updates, and warmup ratio 0.0333. Its saved policy specifies 55 action/state channels, three-camera input, chunk length 50, ten inference steps, and BF16 evaluation with compilation. The inspected configuration has `bias_update_speed=0`, sequence-wise routing loss 10^{-3} and router z-loss 10^{-4}. Thus this run must not be described as validating a loss-free router update. Teacher dependencies and normalization conversion are part of the training burden. Both the released post-trained checkpoint and the locally post-trained checkpoint are evaluated; the foundation pre-train itself is inherited, not reproduced.

Table 11. LingBot-VLA 2.0 reproduction settings. The local run changes the adaptation budget; it does not repeat the foundation pre-train.

Setting	Observed configuration
Initialization / backbone	Released LingBot foundation / Qwen3-VL-4B plus sparse action experts
Input / native representation	Three cameras; 55 state/action channels
Optimizer / local batch	Muon; micro-batch 32, global batch 96
Adaptation / training GPUs	50,000 updates; three H200 GPUs
Warmup / terminal base LR	Ratio 0.0333 / 5×10^{-5}
Action chunk / integration	50 predicted/executed actions; ten flow steps
Completed evaluation	Clean; unseen instructions; seed 0; BF16 and compilation
Coverage / rollout slots	50 tasks $\times$ 100 episodes; four GPUs $\times$ three slots

3.3.3 RoboTwin Success Results

The two original `stats.txt` files each contain exactly 50 completed tasks, 100 episodes per task, and zero skipped tasks. The released checkpoint achieves $4486/5000 = 89.72\%$ Clean success; the local checkpoint achieves $2889/5000 = 57.78\%$. Randomized evaluation is absent from these completed records. Exact counts take precedence over the rounded 89.7% and 57.8% in the source summaries.

Table 12. LingBot-VLA 2.0 closed-loop results on its common local evaluator. Missing Randomized outcomes remain unevaluated.

Metric	Official weights	Local 50k checkpoint
Clean successes / episodes	4486/5000	2889/5000
Clean success rate	89.72%	57.78%
Randomized success rate	–	–
Completed / skipped tasks	50 / 0	50 / 0
Local minus official	-31.94 percentage points on Clean	

3.3.4 Training Resources and Rollout Runtime

The final training progress interval is 58:07:18 on three GPUs, giving **174.365 H200 GPU-hours**. This excludes setup and the final asynchronous export; it is a scoped training interval, not the total cost of developing the reproduction. The final HF export completes successfully. The six weight shards occupy **25.504 decimal GB** for either checkpoint. This is the saved export size, not BF16 resident inference memory. The local evaluation takes 32,130 seconds (8.925 h) with four GPUs and 12 slots; the official-checkpoint evaluation takes 27,660 seconds (7.683 h). These exact totals replace a source note that loosely described both runs as about 8.9 h.

Local sample exposure is $96 \times 50{,}000 = 4.8$ million windows. The source reference recipe reports batch 1024 and 50k updates, or 51.2 million windows: a factor of 10.67 more. At batch 96, matching that count would require about 533,334 updates. This arithmetic is a budget conversion, *not* a prediction of the resulting score. Larger batch, learning-rate warmup, data conversion, teacher paths, precision, and initialization can also affect the result. The comparison is consistent with insufficient adaptation, but does not isolate exposure as the sole cause. The three observations from LingBot and ABot do not constitute a controlled exposure–performance scaling curve.

3.3.5 Training Convergence

The new log extraction retains all 50,000 update-level records. Logged total loss falls from 0.3906 at update one to 0.0194 at update 50,000, while the VLA component changes from 0.3594 to 0.0149. The depth/video losses and learning-rate series are retained in the data CSV. These optimization observations establish learning progress under the native objective; the independently measured 57.78% Clean result determines the checkpoint's task capability. Neither low final loss nor the local loss slope predicts recovery to the official success rate.

3.3.6 Task-Level Profiles and Failure Diagnosis

The largest local failures include plastic-box can placement (2/100), block handover (5/100), and dual-bottle picking (10/100), while roller grasping reaches 100/100. Exact task identifiers and all counts appear in Appendix D. This unevenness is useful for selecting diagnostic tasks. It does not prove a particu-

Table 13. LingBot resource observations. Training time is the final progress interval; evaluation time includes the complete task queue. Artifact GB do not measure resident BF16 VRAM.

Metric	Official weights	Local 50k checkpoint
Reference / local sample exposure	51.2M (source recipe)	4.8M
Measured adaptation wall time	–	58:07:18
Measured H200 training GPU-hours	–	174.365
Clean evaluation wall time	7.683 h	8.925 h
Clean evaluation GPU-hours	30.733	35.700
Primary weight artifact	25.504 GB	25.504 GB
Inference peak VRAM / p95 latency	– / –	– / –

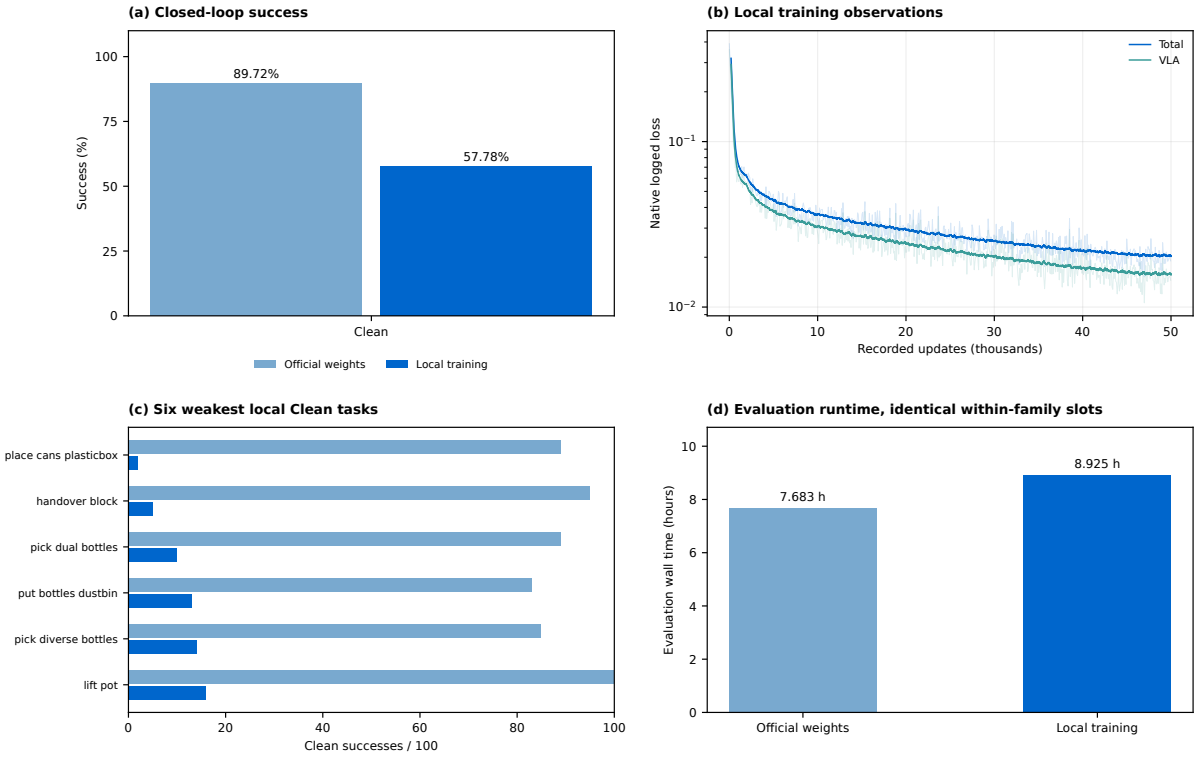


Figure 11. LingBot-VLA 2.0 experimental results. (a) Official and local Clean outcomes use 5,000 episodes each. (b) Total and VLA loss come from all 50,000 update records; light traces display every 83rd record and dark traces use a 200-observation moving average. (c) The six lowest local Clean tasks are compared with the official checkpoint under the same within-family evaluator. (d) Full evaluation queue time uses the same four-GPU, 12-slot configuration. Randomized outcomes and serving latency are unavailable.

lar teacher, camera layout, or MoE component caused a gain or failure. The 31.94-point official-to-local Clean gap establishes substantial recipe sensitivity under the two available checkpoints. A matched exposure study and teacher/MoE ablations are needed to quantify mechanism value. The current result validates the local training-to-export-to-rollout chain, and provides a checkpoint to improve; it does not reproduce the official checkpoint's capability level.

Table 14. Ten lowest local LingBot Clean task results. Each count is out of 100; the difference is local minus official in percentage points. All 50 tasks remain in Appendix D.

Task	Official	Local	Difference
place_cans_plasticbox	89/100	2/100	-87
handover_block	95/100	5/100	-90
pick_dual_bottles	89/100	10/100	-79
put_bottles_dustbin	83/100	13/100	-70
pick_diverse_bottles	85/100	14/100	-71
lift_pot	100/100	16/100	-84
move_can_pot	88/100	16/100	-72
scan_object	96/100	16/100	-80
click_bell	58/100	18/100	-40
move_stapler_pad	70/100	33/100	-37

4 Empirical Evaluation of World–Action Models

This chapter supplies the WAM evidence for the proposed evaluation framework, examining how predictive representations, inherited priors and task conditions affect capability and resource interpretation. OpenWAM official/local comparisons measure adaptation outcomes and costs; LiLa-WAM tests the practical scope of compact latent prediction with visual task selection. Implementation inspection identifies which mechanisms are executable, and the training and rollout audits establish which research-value and reproduction-evidence criteria are satisfied. Each method's setup, results and diagnosis feeds the dimension profiles in Chapter 5.

World–action modeling. A world–action model (WAM) learns representations of future observations together with a robot action policy. Given the current observation o_t, task condition c and robot state q_t when available, its coupled prediction can be written schematically as

$$(\hat{z}_{t+1:t+K}, \hat{A}_{t:t+H-1}) = F_\theta(o_t, c, q_t), \tag{4}$$

where z denotes a future visual representation and A an action chunk. This notation summarizes the prediction targets; it does not require both outputs to be decoded at deployment. Future representations can retain video structure, as in OpenWAM, or occupy a compact feature space, as in LiLa-WAM. Predictive supervision aims to connect visual change with controllable motion. Its contribution must be tested by controlled ablation rather than inferred from a success score alone.

Shared reproduction protocol. For each WAM, we identify the inherited visual or video prior, frozen modules, task condition, prediction targets, action representation and execution adapter. We then report local adaptation settings, checkpoint-specific rollout outcomes and measured resource costs. OpenWAM uses text conditioning and an end-effector interface; the inspected LiLa-WAM uses task-name-selected visual transition tokens and joint actions. These differences remain explicit in comparisons. The two methods retain Sections 4.1 and 4.2, respectively.

4.1 OpenWAM

4.1.1 World–Action Coupling and Interfaces

OpenWAM develops a modular world–action modeling stack with dedicated world and action capacity and explicit interaction between them(Wang et al., 2026). The inspected run combines Wan2.2-TI2V-5B, Wan VAE and ActionDiT in a dual-system joint-self-attention architecture. VAE and T5 text encoding are frozen. Training uses three views, 33-frame windows with video stride four, and configured height/width 384×320. Twenty physical end-effector action channels map to slots 0–9 and 34–43 in an 80-channel unified representation. This differs materially from ABot's joint-action interface. Inverse kinematics and the execution adapter are therefore part of the comparison, as are the visual inputs and foundation initialization.

4.1.2 Experimental Setup

The local run starts from the released Foundation checkpoint (step 154000), rather than training the video prior or robotic foundation from scratch. It uses four H200 NVL GPUs, BF16, ZeRO-2, AdamW, global batch 64, 80k updates, learning rate 10^{-4}, 5% warmup, and equally weighted video/action losses. The saved `max_steps=80000` and the final checkpoint establish the stopping point; the `num_epochs=1`

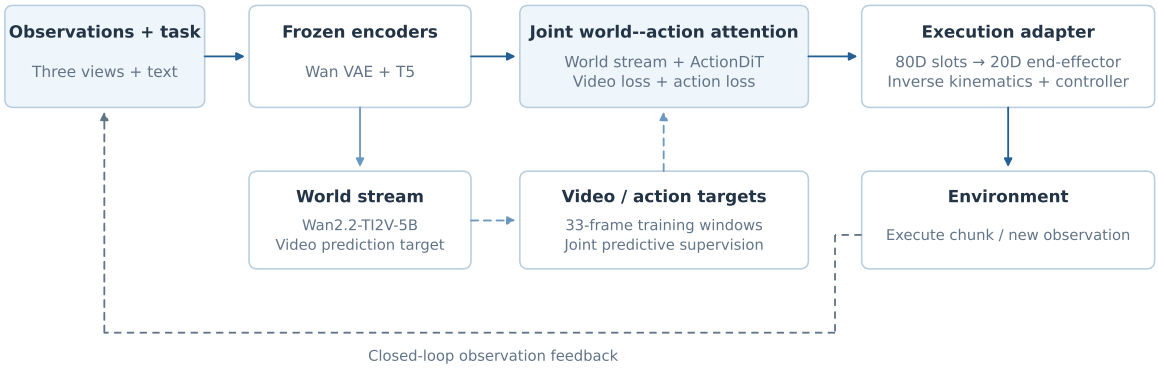


Figure 12. OpenWAM reproduction interfaces. Frozen Wan VAE and T5 encoding condition a Wan2.2 world stream and ActionDiT action stream coupled by joint self-attention. Training supervises video and action targets; the execution adapter selects 20 physical end-effector channels from the 80-channel representation.

field alone does not describe the effective training duration. The final reported total/video/action losses are 0.08358/0.08310/0.00048. Their numerical magnitudes cannot be compared with ABot's or LingBot's losses because targets, normalization and loss definitions differ.

Table 15. OpenWAM local adaptation and evaluation settings. The released Foundation initialization is inherited.

Setting	Observed configuration
World / action modules	Wan2.2-TI2V-5B plus Wan VAE / ActionDiT
Training views / video window	Three views; 33 frames, stride four; 384 × 320
Native action / unified channels	20D end-effector commands / 80D slots
Updates / local global batch	80,000 / 64 (four GPUs × 16)
Optimizer / learning-rate schedule	AdamW; 10^{-4}, cosine, warmup ratio 0.05
Training precision / parallelism	BF16 / ZeRO-2 on four H200 NVL GPUs
Evaluation scenes / instructions	Clean and Randomized / unseen, seed 0
Coverage per scene / slots	50 tasks × 100 episodes / four GPUs × four slots
Observation and success checks	Observation at chunk start; success checked after every action

4.1.3 RoboTwin Success Results

Two checkpoints each have 100 original task/scene result files, covering all 50 tasks in both scenes. The reported rates are exact multiples of 1/100; reconstructed successes sum to the source totals in Table 16.

Table 16. OpenWAM closed-loop outcomes. Official and local checkpoints share the adapted OpenWAM evaluator and each cover 10,000 episodes.

Metric	Official weights	Local BS64-80K
Clean successes / episodes	4701/5000	4565/5000
Clean success rate	94.02%	91.30%
Randomized successes / episodes	4683/5000	4570/5000
Randomized success rate	93.66%	91.40%
Two-scene mean	93.84%	91.35%
Local minus official	−2.49 percentage points overall	

The released OpenWAM checkpoint achieves 93.84% across 10k episodes. The local BS64-80K checkpoint reaches 91.35%, with 91.30% Clean and 91.40% Randomized success. The matched local evaluator gives

a **2.49-point adaptation gap**. Local exposure is 5.12 million video windows, versus a source reference estimate of $118655 \times 256 = 30.376$ million, or 16.86%. That estimate combines the released update count with the reported reference global batch; it does not establish the official GPU count or monetary cost. A video window and an ABot training example contain different information and compute, so these counts are not interchangeable units of sample efficiency.

4.1.4 Training Resources and Rollout Runtime

The recorded 65:31:01 local training interval gives **262.068 H200 GPU-hours**; the saved checkpoint is **24.814 GB**. The local evaluator requires 17:03:02 for Clean and 16:42:39 for Randomized (33:45:41 total). Four allocated GPUs imply 135.046 evaluation GPU-hours, roughly 52% of the recorded downstream training budget. The corresponding official-weight evaluation takes 29:10:05 (116.672 GPU-hours). Identical model structure with different wall time illustrates why rollout duration cannot be treated as pure neural-network latency. Success-dependent episode length, CPU load, rendering and contention can all change runtime.

Table 17. OpenWAM resource observations. Downstream training excludes Foundation pre-train; evaluation-device hours assume the four recorded GPUs are allocated throughout each queue.

Metric	Official weights	Local BS64-80K
Reference / local sample exposure	30.376M (source recipe)	5.120M
Measured adaptation wall time	–	65:31:01
Measured training GPU-hours	–	262.068
Clean evaluation wall time	14:04:28	17:03:02
Randomized evaluation wall time	15:05:37	16:42:39
Total evaluation GPU-hours	116.672	135.046
Primary weight artifact	24.814 GB	24.814 GB
Inference peak VRAM / p95 latency	– / –	– / –

Both OpenWAM evaluations use commit `0aeea2d` and an adapted execution path: observation and policy requests occur at the first step of an action chunk, while success and step limits are checked after every executed action. Four independent services per GPU provide 16 slots across four GPUs. The two OpenWAM checkpoints share this protocol, supporting their internal comparison. This does not establish equivalence to the upstream evaluator or to ABot's recorded commit `13c3c47`. The small Clean–Randomized difference is evidence about the tested randomized distribution, which is included in adaptation data; it is not a clean-only-to-randomized generalization test.

4.1.5 Training Convergence

The console supplies repeated rank/progress-bar records. For an auditable display we retain the first occurrence at each positive progress counter, yielding 80,000 observations. The displayed total/action losses change from 0.2679/0.0537 near the start to 0.0836/0.0005 at the end; video loss remains the dominant numerical contribution. These are four-decimal console values. The more precise final tracker values, 0.08358/0.08310/0.00048 for total/video/action, are retained separately and are not substituted throughout the curve. The progress display can lead or lag the associated optimizer update by one because the logger sets its postfix before advancing its counter; this precision is sufficient to visualize convergence, not to infer a particular update's causal effect.

4.1.6 Task-Level Profiles and Failure Diagnosis

Table 18 follows the ten lowest local Clean task rates and retains both scenes. Local Clean and Randomized averages are nearly equal, but the task-specific gaps differ; an overall mean can hide substantial task deficits. For instance, the local `turn_switch` results are 45/100 and 57/100, while `hanging_mug` falls to 36/100 and 34/100. Their weakness is observed in both scene distributions rather than confined to randomized rendering.

4.1.7 Algorithmic and Practical Value

OpenWAM supplies the highest observed two-scene rate among the locally trained methods in this snapshot. Relative to the ABot Qwen3.5-4B record, its local rate is 5.85 points higher and its recorded down-

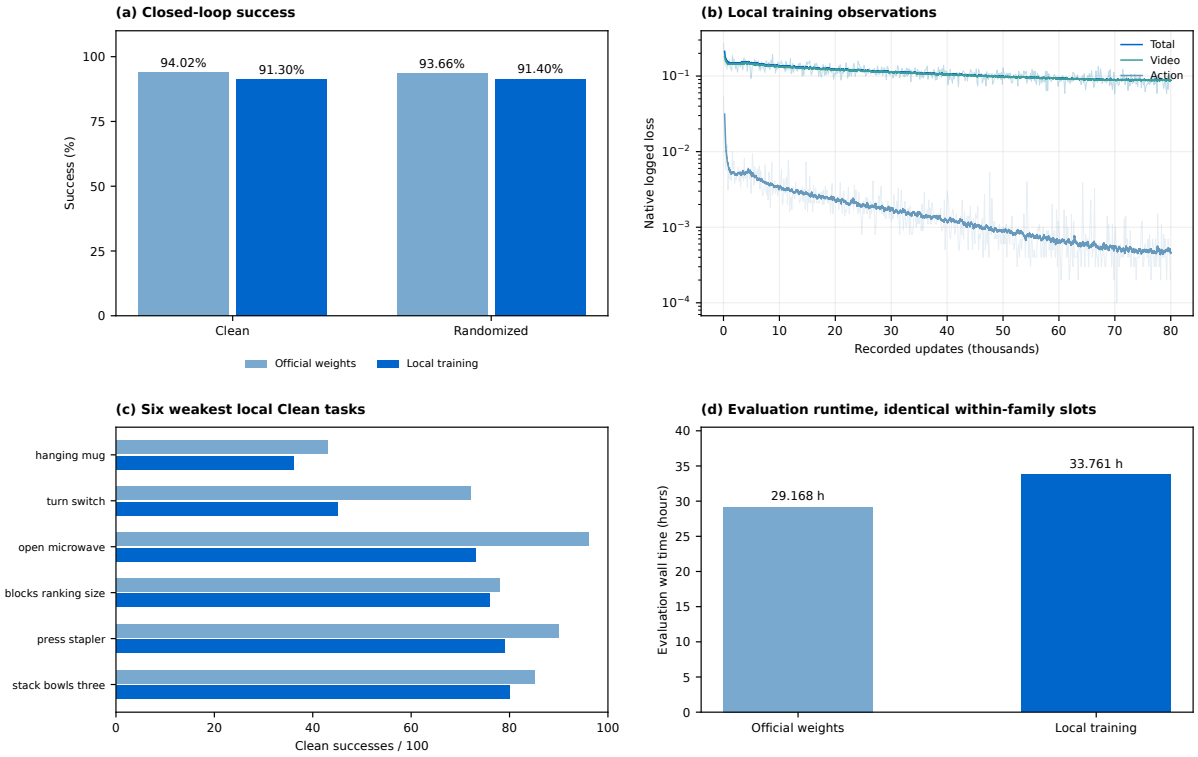


Figure 13. OpenWAM experimental results. (a) Separate Clean and Randomized rates use 5,000 episodes per scene and checkpoint. (b) Native total, video and action loss use 80,000 selected console observations, every 133rd raw value, and a 200-observation moving average; console losses are rounded and progress counters have a possible one-update offset. (c) Six weakest local Clean tasks retain the same within-family checkpoint comparison. (d) Evaluation wall time covers both scenes on four GPUs and 16 slots; it is not policy inference latency.

Table 18. Ten lowest local OpenWAM Clean tasks. Each C/R entry is a pair of success counts out of 100 per scene; differences are local minus official two-scene means in percentage points.

Task	Official C/R	Local C/R	Difference
hanging_mug	43/54	36/34	-13.5
turn_switch	72/65	45/57	-17.5
open_microwave	96/73	73/71	-12.5
blocks_ranking_size	78/77	76/83	+2.0
press_stapler	90/94	79/91	-7.0
stack_bowls_three	85/80	80/82	-1.5
pick_diverse_bottles	83/84	81/88	+1.0
place_can_basket	78/76	81/79	+3.0
move_stapler_pad	98/98	83/91	-11.0
put_object_cabinet	88/92	85/75	-10.0

stream GPU-hours are about 33.5% lower. These are useful procurement observations under the available recipes, but do not isolate an architectural advantage: initialization, data windows, observations, action spaces, exposure, kernels and evaluator paths differ. Training a video foundation is outside the reported 262.068 GPU-hours, and no full-lifecycle cost advantage has been demonstrated.

Within OpenWAM, the largest average local deficits occur on `turn_switch` (-17.5 points), `hanging_mug` (-13.5), `open_microwave` (-12.5) and `move_stapler_pad` (-11.0). These deficits occur across both domains and motivate investigating contact and alignment behavior. The available experiment changes an entire training recipe; it does not test whether explicit future prediction, joint attention or the video loss individually improves control. Such claims remain architectural hypotheses locally, even when supported by the original authors' experiments.

4.1.8 Additional Video-Backbone Training Prefix

The October 28 inspection also finds an OpenWAM Wan2.1-VACE-1.3B run configured for 80k updates, global batch 64 and the same 80-channel action representation. The latest saved trainer state identifies update 50,000; the console reaches displayed progress 51,180/80,000. The saved 50k artifact is 17.211 GB. Its final displayed total/video/action losses are 0.0284/0.0275/0.0009, with a displayed interval around 71:04:28. These values describe an unfinished training prefix and native rounded logging, not a complete 80k adaptation cost. No corresponding task-result set was found, so this candidate receives no capability score and is recorded outside the 12 evaluated checkpoint profiles. Its source configuration and trainer state are retained in the refresh audit.

4.2 LiLa-WAM

4.2.1 Compact Latent Prediction and Task Specification

LiLa-WAM couples future-feature prediction and action generation using a frozen DINOv3 ViT-L/16 and a compact action model(Yang et al., 2026). The source describes about 0.5B total and 0.2B trainable parameters. These are source-reported scale descriptors, not a local enumeration of all resident tensors. The inspected setting uses a head image at 320×240, 14 joint-action channels, 32 predicted actions, execution of the first 16, ten flow-matching steps, and spline smoothing.

The policy is task-conditioned through visual tokens. The inspected `set_task(task_name)` loads a precomputed `task_cond.npy`; `_predict_chunk` explicitly leaves its instruction argument unused. Consequently, an evaluator labeled `unseen` does not establish understanding of unseen natural-language instructions in this run. VTT replaces language with a known task's visual transition vector. This can reduce runtime and training burden for a fixed task inventory, but requires constructing and distributing task conditions and does not provide an instruction interface for arbitrary new requests. Its operational value must be judged against that requirement.

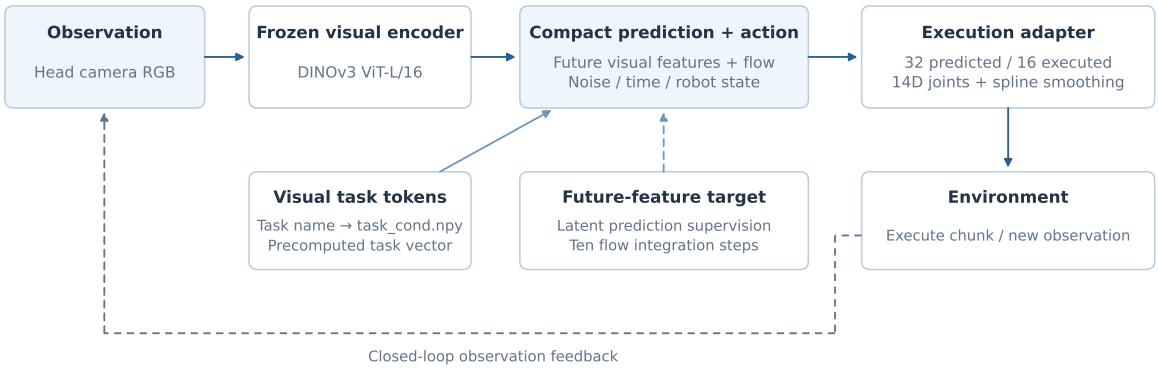


Figure 14. LiLa-WAM reproduction interfaces. A frozen DINOv3 encoder provides head-camera features. A task name selects precomputed visual transition tokens; the inspected policy does not use the instruction argument. Compact future-feature prediction and action generation share the task condition. The controller executes 16 of 32 predicted actions with spline smoothing.

4.2.2 Experimental Setup

4.2.3 RoboTwin Success Results

The official weights achieve $224/250 = 89.60\%$ Clean success; the local stage-2 epoch-4 weights achieve $222/250 = 88.80\%$. Each evaluation covers the same 50 task names with five episodes per task and four workers. All worker JSON counts match their per-task final log counters. The local evaluation setting resolves to the saved stage-2 checkpoint from October 23. The official results are retained in a separate backup directory because the active worker-result filenames were reused by local evaluation.

The local-minus-official difference is -0.80 percentage points, or two successes across 250 episodes. Nineteen tasks change their success count; 31 are unchanged. Thirty-two official and 33 local tasks achieve 5/5, while seven official and five local tasks fall below 4/5. Aggregate similarity can therefore coexist with task-level changes. Five episodes give coarse 20-point per-task increments; these observations do not prove statistical equivalence or isolate the latent-prediction mechanism. Randomized results remain unavailable for both checkpoints.

Table 19. LiLa-WAM settings shared by the official-weight and local stage-2 evaluations. Its visual task interface is explicit.

Setting	Observed configuration
Visual encoder / policy scale	Frozen DINOv3 ViT-L/16 / approximately 0.5B (source)
Task specification	Task-name-selected precomputed visual transition vector
Input view / size	Head camera / 320 × 240
Action / internal state channels	14 / 16
Prediction / execution horizon	32 / 16 actions
Integration / smoothing	Ten flow steps / B-spline
Completed checkpoints	Official stage-2 weights and local stage-2 epoch four
Evaluation scenes / coverage	Clean / 50 tasks × five episodes
Hardware / slots	One H200 NVL / four parallel workers

Table 20. LiLa-WAM official/local Clean results. Each column represents a complete 250-episode evaluation, with task-name-selected visual conditions.

Metric	Official weights	Local stage-2 epoch 4
Clean successes / episodes	224/250	222/250
Clean success rate	89.60%	88.80%
Working Wilson 95% interval	85.20%–92.80%	84.29%–92.14%
Randomized success rate	–	–
Tasks with five successes	32/50	33/50
Tasks below four successes	7/50	5/50
Local minus official	−0.80 percentage points	

The working pooled Wilson intervals describe episode sampling under a fixed tested task mixture. They exclude training-seed variation, task-population uncertainty and protocol bias. Overlapping intervals and proximity to the source's 90.48% are not equivalence tests. The completed local evaluation establishes that the saved local training prefix can execute this task protocol; the contribution of individual components still requires controlled ablation.

4.2.4 Local Training Convergence and Saved Prefixes

Stage-1 checkpoints extend through epoch 12 and stage-2 checkpoints through epoch 4. The October 26 local rollout now verifies the epoch-4 stage-2 checkpoint, superseding the previous pending status. The stage-2 training log enters epoch 5; successful evaluation of epoch 4 does not establish completion of the configured 40-epoch schedule. The saved learning rate is 4×10^{-5} while `lr_min` is 5×10^{-5}, and the log warns that this schedule increases rather than decreases the rate. The observed 88.80% is tied to that actual recipe; the effect of correcting the schedule remains untested.

Loss extraction is restricted to the completed saved prefixes: 23,643 observations over stage-1 epochs 1–12 and 7,881 over stage-2 epochs 1–4. Seven observations from partial stage-1 epoch 13 and four from partial stage-2 epoch 5 are excluded and counted in the audit. The update axis is reconstructed from the epoch, batch index and 39,406 minibatches per epoch; counters restart for stage 2. Stage-1 logged action MSE changes from 0.3473 to 0.0014; stage 2 starts at 0.0019 and its last selected record is 0.0007. These are component losses at sampled updates. The epoch average is the full training objective, which also includes feature supervision; it is not the same scalar as action MSE.

Table 21. LiLa local training prefixes. Summed epoch timers exclude startup, scans, checkpoint export and partial epochs. Local rollout is now verified, but the epoch timers lack an independent device-allocation trace and are not inserted into the H200 cost score.

Training prefix	Saved epochs	Epoch timers (h)	Last epoch total loss
lila_stage1	12	57.35	0.0689
lila_stage2	4	20.59	0.0641

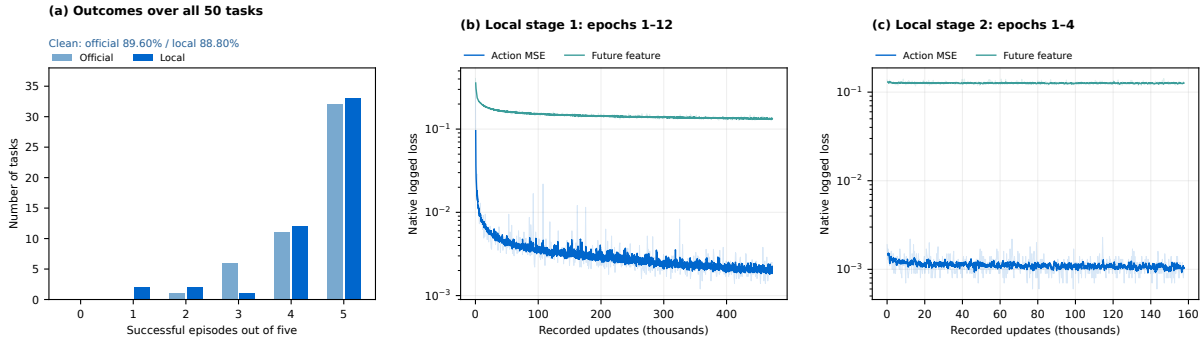


Figure 15. LiLa-WAM experimental results. (a) Official/local task-outcome distributions over 50 tasks with five episodes each; compact text reports Clean success, 89.60%/88.80%. (b,c) Local action-MSE and future-feature curves cover saved stage-1 epochs 1–12 and stage-2 epochs 1–4, with separate update axes. Light curves subsample every 39th/13th observation; dark curves use 20-observation moving averages. The local rollout uses the stage-2 epoch-4 checkpoint reached by these training prefixes.

4.2.5 Training and Deployment Resource Boundaries

The published single-24-GB-GPU feasibility and approximately 110 RTX-5090 GPU-hours in the repository are *source claims*, not locally measured H200 training costs. They are therefore not entered into the local cost score. The official action checkpoint is loaded separately from DINOv3, and local training checkpoints can contain optimizer state. A single checkpoint's file size would undercount required dependencies or count training-only tensors; no deployment-package score is assigned until a clean inference bundle is measured. One H200 with four workers proves feasibility on that machine, not minimum VRAM or a robot's real-time deadline.

4.2.6 Task-Level Profiles and Evidence Limits

Table 22. Ten lowest local LiLa-WAM Clean task results, retaining the official-weight comparator. A single extra success changes a task's displayed rate by 20 points; these rows should not be treated as precise task rankings.

Task	Official / 5	Local / 5	Gap (pp)
place_a2b_left	3/5	1/5	-40
place_dual_shoes	4/5	1/5	-60
move_stapler_pad	3/5	2/5	-20
open_microwave	4/5	2/5	-40
place_mouse_pad	4/5	3/5	-20
click_bell	5/5	4/5	-20
hanging_mug	3/5	4/5	+20
lift_pot	5/5	4/5	-20
pick_diverse_bottles	3/5	4/5	+20
place_a2b_right	5/5	4/5	-20

The lowest local tasks include `place_a2b_left` and `place_dual_shoes` at 1/5, and `open_microwave` at 2/5. Conversely, `stamp_seal` rises from 2/5 with official weights to 5/5 locally. Such changes motivate larger matched evaluations and failure replay, rather than a precise task ranking from five episodes. High success on known task vectors does not demonstrate free-form instruction understanding. The next extension is to increase identical scene sampling for both checkpoints and add Randomized coverage.

5 The Proposed RoboSeed Evaluation Framework

5.1 Decision Unit, Eligibility, and Interpretation

We now formalize the framework introduced in Section 1 and apply it to the evidence collected in Chapters 3 and 4. Our goal is to give community researchers a reusable assessment of task performance, resource feasibility and experimental support. The five dimensions answer complementary questions; the preference settings and eligibility rules connect them to a specific research decision.

A scientifically useful assessment must distinguish *what a checkpoint can do*, *what it costs*, and *what the experiment establishes about its mechanism*. A single success leaderboard conflates these questions. We

assess every reproduced policy family and every available RoboTwin checkpoint, including incomplete candidates. The unit is a **checkpoint–training recipe–interface–evaluator tuple**; the PX harness is a systems intervention on an existing unit. Broader ABot results supply transfer evidence and diagnostic context, but LIBERO percentages, RoboCasa percentages and CALVIN chain lengths are not averaged into RoboTwin capability.

Five dimensions, each on a 0–100 scale, define the profile: capability P, downstream training affordability T, deployment affordability D, locally verified research value V, and reproduction evidence R. Larger values are preferable under the stated rules. This rubric is a **versioned decision policy**, not an externally validated universal measure of algorithm quality. Thresholds and weights express explicit budget preferences; experimental counts and measured costs supply the empirical inputs. We state both, retain the raw metrics, and publish sensitivity to alternative preferences.

Three comparability gates precede a choice. First, the task interface must meet the application requirement: VTT task selection and free-form language are different capabilities. Second, training scope must be consistent: downstream adaptation of a released foundation does not include its inherited pre-train cost. Third, a causal architecture ranking requires matched datasets, input/actuation interfaces, precision, execution semantics, hardware and multiple training seeds. Our cross-method matrix does not pass the third gate. Scores can guide follow-up and budget screening; cross-method differences cannot be interpreted as isolated architecture effects.

5.2 Quantitative Scoring Rules

5.2.1 Capability and Sampling Uncertainty

For the shared RoboTwin target, let p_C and p_R be task macro-average success fractions in Clean and Randomized. Define

$$P = 100\,(p_C + p_R)/2. \tag{5}$$

All observed scenes have equal denominators per task, so the macro- and micro-averages coincide there. Both scene results are required to compute P. If either scene is missing, P is shown as a dash (–) rather than a numerical range or an imputed value. LingBot official and LiLa official therefore have no two-scene P score, while their observed *Clean-only* values remain 89.72% and 89.60% in the method-specific result tables. A completed training run without rollout outcomes also receives a dash for P.

Episode sampling uncertainty is calculated only from observed outcomes. Machine-readable output includes a working Wilson 95% interval for each observed scene; for s successes in n episodes, with $\widehat{p} = s/n$ and $z = 1.96$, it uses

$$\frac{\widehat{p} + z^2/(2n) \;\pm\; z\sqrt{\widehat{p}(1-\widehat{p})/n + z^2/(4n^2)}}{1 + z^2/n}. \tag{6}$$

Pooling is only a working binomial approximation for a fixed tested task mixture. Task heterogeneity, shared seeds, simulator nondeterminism and training variation can violate its assumptions. We do not use overlapping intervals to prove equivalence, or small point differences to establish superiority. No multi-seed training uncertainty is estimated from one checkpoint.

5.2.2 Training Affordability and Lifecycle Scope

For a positive cost x, use a fixed-anchor log utility

$$F(x; a, b) = 100\ \mathrm{clip}_{[0,1]}\left(\frac{\log(b/x)}{\log(b/a)}\right), \quad 0 < a < b. \tag{7}$$

A cost at or below a earns 100, a cost at or above b earns zero, and multiplicative changes have equal utility increments between these anchors. Anchors remain fixed when another method is added, avoiding sample-dependent min–max rankings. A zero score means exceeding a chosen budget anchor, not scientific worthlessness.

Training score is

$$H = \sum_j N_j t_j, \qquad T = F(H; 64, 1024), \tag{8}$$

where N_j is the allocated H200 GPU count and t_j is the recorded training interval in hours. The anchors represent 16 hours and 256 hours on four GPUs; within this range each doubling costs 25 score points. All *scored* local intervals are on H200 hardware. An RTX-5090 hour is not substituted for an H200 hour. Official checkpoints with no measured reference training runtime receive a dash for T, even when reference updates and batch are known.

T measures *the observed downstream recipe's device cost*, not compute to a common target success rate. The stopped LingBot run can therefore have a favorable T and a poor P simultaneously; this is why T must never be read alone. It is not time-to-90%, unique-data efficiency, FLOPs, energy, utilization or price. Approximate legacy training memory is retained in Table 5, but is not used in T because sampling conventions and units are not uniform. A candidate that exceeds an actual GPU's memory capacity is infeasible regardless of its score.

For a deployment project, a more complete cost is

$$C_{\text{project}} = C^{\text{allocated}}_{\text{inherited pretrain}} + C_{\text{adapt}} + C_{\text{search/failures}} + KC_{\text{evaluation}} + C_{\text{serving}} + C_{\text{integration}}. \tag{9}$$

Only selected terms are measured here. In particular, foundation pre-train is inherited for LingBot and OpenWAM, VLM/vision priors are inherited by ABot and LiLa, failed trials are not comprehensively metered, and engineering person-hours are unavailable. Hence neither a dollar-denominated total cost nor an end-to-end training-cost winner is reported.

5.2.3 Deployment Affordability and Required Measurements

Deployment has distinct storage, memory and responsiveness costs. Define

$$D = 0.2F(W; 2, 32) + 0.4F(M; 8, 80) + 0.4F(L_{95}; 50, 1000), \tag{10}$$

where W is the primary released model-weight artifact size in decimal GB, M is warmed policy inference peak memory in GiB, and L_{95} is p95 policy-request latency in milliseconds. The respective anchors encode a 2–32 GB distribution budget, 8–80 GiB serving budget and 50–1000 ms request budget. These are declared assessment preferences, not robotics safety deadlines. The primary artifact term measures distribution burden only; separately required encoders, teachers or caches must also be inventoried before procurement. It does not establish the complete installation footprint.

A valid latency measurement must record the same accelerator, precision, batch one, warmup and request count, camera inputs, executed/predicted horizon, integration steps and whether preprocessing/transfer/decoding are included. Latency should include those policy-request components while excluding simulator reset and physics. Sampling several hundred warmed requests with raw timings is required before scoring M and L_{95}; simulator throughput, a mean request time or a nominal parameter count is insufficient. A robot-specific requirement must additionally compare latency to its control deadline and open-loop action horizon.

No policy in this snapshot has the required paired memory and p95 trace, so every D entry is a dash. Only the primary-artifact component is available for ABot, LingBot and OpenWAM; its measured bytes and utility are retained in the raw metadata and subscores, without presenting that partial component as the full D score. LiLa's checkpoint packaging is not comparable. Training VRAM is not substituted for inference VRAM, and a frozen encoder still incurs inference cost. A small parameter count motivates a future measurement but does not supply a measured latency score.

5.2.4 Locally Verified Algorithmic Value

Research value is evaluated through the strength of *locally established, reusable conclusions*, rather than novelty adjectives or paper reputation. Four yes/no criteria each contribute 25 points:

1. **Executable mechanism:** an inspected implementation and effective configuration identify the proposed mechanism and its conditioning/actuation path.
2. **Decision comparator:** local outcomes include a documented within-family reference or controlled alternative that supports a concrete decision, with confounds disclosed. An unrelated published percentage is insufficient.
3. **Mechanism isolation:** a local component ablation holds data, exposure, interfaces and evaluator fixed and is repeated over at least three training seeds. A whole-checkpoint comparison does not qualify.
4. **Transfer test:** the same local checkpoint is tested without adaptation on a documented new distribution, with outcomes and denominators. Independently retrained checkpoints on several benchmarks do not alone qualify.

Thus $V = 25\sum_{k=1}^{4} v_k$. A criterion is zero when qualifying local evidence has not been supplied; this evaluates the evidence portfolio, not an unknown future capability. Original-paper ablations motivate hypotheses but are not counted as locally reproduced evidence. The score is assigned at the method-family level and shared across its checkpoints. It must not be taken to imply that every individual backbone has a transfer result.

These scores explain *which claims the reproduction supports*. ABot's extra points reflect the local comparison and robustness portfolio; they do not prove AML is intrinsically more valuable than predictive world

Table 23. Research-value decisions and their local evidence. A one denotes a satisfied criterion. No local mechanism-isolation experiment meets the stated rule.

Family	Impl.	Comp.	Isolate	Xfer.	Evidence and V
ABot-M0	1	1	0	1	Backbone budget comparison; same LIBERO checkpoint on LIBERO-plus. $V = 75$.
LingBot 2.0	1	1	0	0	Official/local recipe gap, common local evaluator; no isolated MoE/teacher effect. $V = 50$.
OpenWAM	1	1	0	0	Official/local adaptation gap on the same execution path; randomized data used in training. $V = 50$.
LiLa-WAM	1	1	0	0	VTT inspected; official/local stage-2 comparison on a documented common evaluator. $V = 50$.

modeling. LingBot and OpenWAM provide testable predictive mechanisms, while LiLa offers a compact, language-free route with a different application boundary. Quantifying their individual contributions requires the missing ablations.

5.2.5 Reproduction Evidence

Five binary checks each contribute 20 points to R: (i) a configuration and weight-to-run mapping exists; (ii) original task records support the reported counts; (iii) every observed scene covers all 50 tasks with at least 20 episodes each; (iv) the local training interval and allocated device count are documented; and (v) at least three independent training seeds have complete evaluations. These are audit milestones, not a probability that a result is correct. Task totals reconstructed from aggregate rates remain weaker than independent episode traces even when check (ii) passes. Episode-level completeness and seed manifests should accompany future audits.

Evaluated local ABot, LingBot and OpenWAM units earn 80; the two latter official-checkpoint units earn 60 because reference training runtime is unavailable. Both LiLa units earn 40: all tasks are covered, but five episodes do not pass (iii), and independently verified hardware-normalized training intervals are unavailable. The newly completed ABot Qwen3.5-2B unit earns 80 like the other evaluated ABot units. None satisfies the multi-seed check. Missing a Randomized scene affects P and score coverage rather than silently failing the per-observed-scene audit rule.

5.3 Aggregation, Missing Data, and Sensitivity

For dimension vector $\mathbf{s} = (P, T, D, V, R)$, the balanced preference is

$$S_{\text{bal}} = 0.30P + 0.20T + 0.20D + 0.15V + 0.15R. \tag{11}$$

Missing measurements are shown as dashes (–). A composite dimension is calculated only when all its required components are available; the overall score is calculated only when every positively weighted dimension is available. Missing values are stored as `null` in JSON, rather than zero or a numerical range. Missing weights are not redistributed to observed terms. Thus all current aggregate scores are dashes because deployment measurements are incomplete. A measured utility of zero and an audited criterion earning zero points remain valid numbers; they are distinct from an unavailable measurement.

Weighted measurement coverage records available inputs without estimating the missing scores. Let c_P be half the number of observed scene rates, c_T indicate an available training-cost measurement, and $c_D = 0.2I_W + 0.4I_M + 0.4I_L$, where each indicator is one when that deployment input is available. The evidence-audit dimensions have $c_V = c_R = 1$, including audited criteria that earn zero points. For profile weights w_j, define $q = \sum_j w_j c_j$. This is the observed fraction of aggregate input weight, not a success probability or statistical confidence. Even when some inputs of a composite dimension are available, that dimension remains a dash until complete. An overall numerical score requires $q = 100\%$; any ranking must additionally meet the interface/protocol gates and account for episode and training uncertainty. No overall numerical ranking is reported in this snapshot.

For a worked example, ABot Qwen3-VL-2B has $P = 83.78$, $H = 214.049$ GPU-hours, $V = 75$ and $R = 80$, with $T = 56.46$. Its 7.766 GB artifact supplies only the storage component of D; inference memory and p95 latency are unavailable. Therefore both D and the overall score are dashes. Available weighted inputs give $q = 0.30 + 0.20 + 0.20 \times 0.20 + 0.15 + 0.15 = 84\%$. This describes evidence coverage and does not assign an overall performance score. The calculation is reproducible from `assessment.json`.

Changing preferences changes input coverage, but cannot supply missing serving measurements. The deployment-constrained profile assigns more weight to the missing deployment inputs, reducing q for most units. All three profiles therefore withhold the overall score. Once the inputs are complete, their

Table 24. Complete assessment of the 12 checkpoint units. Q3-VL and Q3.5 abbreviate the backbones. P is two-scene capability, T training affordability, D deployment affordability, V research value, and R reproduction evidence. A dash denotes unavailable measurements or an incomplete composite. q reports available input weight. Measured scores are rounded only for display.

Checkpoint	P	T	D	V	R	S_{bal}	q
ABot Q3.5-0.8B	80.6	63.9	–	75.0	80.0	–	84%
ABot Q3-VL-2B	83.8	56.5	–	75.0	80.0	–	84%
ABot Q3-VL-4B	85.5	45.7	–	75.0	80.0	–	84%
ABot Q3.5-2B	84.1	56.6	–	75.0	80.0	–	84%
ABot Q3.5-4B	85.5	34.4	–	75.0	80.0	–	84%
ABot Q3.5-9B	83.8	25.2	–	75.0	80.0	–	84%
LingBot 6B official	–	–	–	50.0	60.0	–	49%
LingBot 6B local	–	63.9	–	50.0	80.0	–	69%
OpenWAM official	93.8	–	–	50.0	60.0	–	64%
OpenWAM local	91.3	49.2	–	50.0	80.0	–	84%
LiLa-WAM official	–	–	–	50.0	40.0	–	45%
LiLa-WAM local	–	–	–	50.0	40.0	–	45%

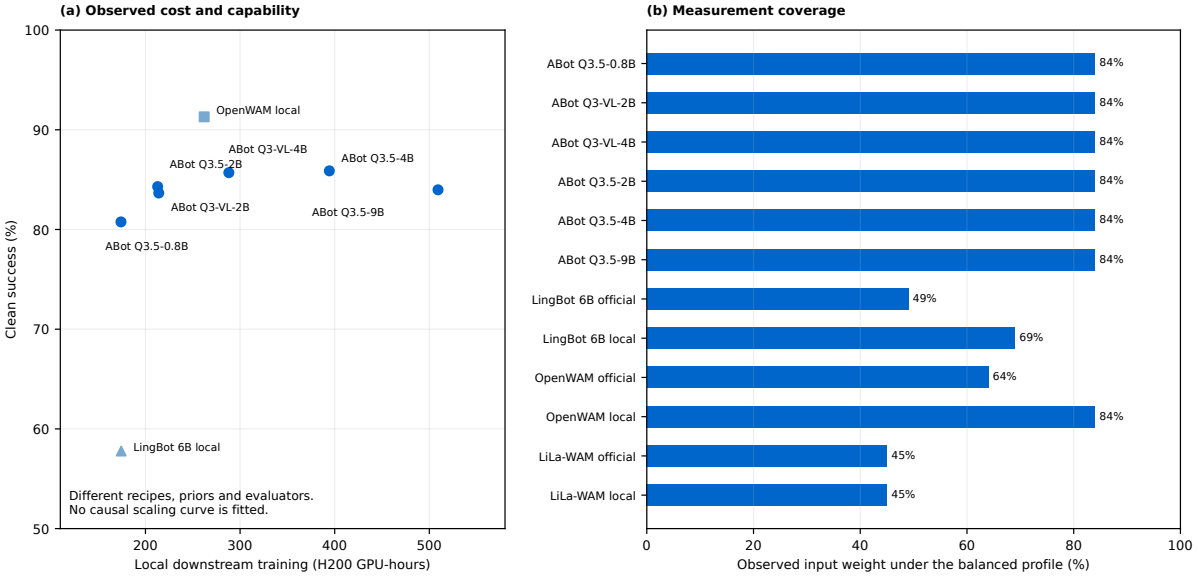


Figure 16. Cost–capability observations and measurement coverage. (a) Only local checkpoints with measured downstream H200 intervals and observed Clean outcomes are plotted. Different initialization, interfaces and protocols prevent a causal scaling interpretation. (b) Bars show available input weight under the balanced profile for all 12 units. Coverage is not an aggregate model score; incomplete scores are shown as dashes in the tables.

Table 25. Preference sensitivity. Weights follow the order (P, T, D, V, R) and sum to one. These are explicit decision preferences rather than empirically learned constants.

Profile	Weights	Use case
Balanced	$(.30, .20, .20, .15, .15)$	General reproduction decision
Training constrained	$(.30, .35, .10, .10, .15)$	Repeated adaptation with limited compute
Deployment constrained	$(.30, .10, .35, .10, .15)$	Serving memory and latency dominate

Table 26. Aggregate availability and weighted measurement coverage under each preference profile. Each cell is aggregate score / q. The aggregate remains a dash until all required inputs are available.

Checkpoint	Balanced	Training constrained	Deployment constrained
ABot Q3.5-0.8B	– / 84%	– / 92%	– / 72%
ABot Q3-VL-2B	– / 84%	– / 92%	– / 72%
ABot Q3-VL-4B	– / 84%	– / 92%	– / 72%
ABot Q3.5-2B	– / 84%	– / 92%	– / 72%
ABot Q3.5-4B	– / 84%	– / 92%	– / 72%
ABot Q3.5-9B	– / 84%	– / 92%	– / 72%
LingBot 6B official	– / 49%	– / 42%	– / 47%
LingBot 6B local	– / 69%	– / 77%	– / 57%
OpenWAM official	– / 64%	– / 57%	– / 62%
OpenWAM local	– / 84%	– / 92%	– / 72%
LiLa-WAM official	– / 45%	– / 40%	– / 40%
LiLa-WAM local	– / 45%	– / 40%	– / 40%

aggregate scores can be compared under the stated weights. Threshold sensitivity should also be assessed if a target budget differs from our anchors; the released scoring function makes those preferences editable. Neither the balanced weights nor the anchor choices are fitted to reproduce a desired method ranking.

5.4 Scientific Conclusions and Practical Choices

The completed 2B result refines the observed budget frontier. Among the matched ABot runs, 0.8B remains the lowest-cost option at 80.63% and 174.1 H200 GPU-hours. Qwen3.5-2B reaches 84.12% at 212.9 GPU-hours, and Qwen3-VL-4B reaches 85.52% at 288.1 GPU-hours. Relative to Qwen3-VL-2B (83.78%, 214.0 hours), the Qwen3.5-2B point estimate is slightly better at slightly lower cost; the 0.34-point difference requires multi-seed validation. Both 4B variants exceed the 9B run's observed success while using fewer GPU-hours. These observations define a recipe-level point-estimate frontier, not a seed-robust architecture ranking.

A capability floor should precede cost optimization. An illustrative 200-H200-GPU-hour adaptation budget and an observed two-scene success floor of 80% admits ABot 0.8B. A 250-hour budget and 84% floor admits the newly completed Qwen3.5-2B candidate; a 300-hour budget and 85% floor admits Qwen3-VL-4B. Qwen3-VL-2B also remains feasible under a 250-hour, 83% requirement. These are screens using observed checkpoints, not guarantees for a new training seed or an untested task mixture.

OpenWAM is a strong candidate for a harmonized follow-up. It combines 91.35% observed two-scene success with 262.068 recorded downstream GPU-hours, but deployment traces and a matched ABot evaluator are missing. Its strong recipe-level result justifies testing the same seed/instruction manifest and common control interface, followed by joint video/action ablations. It does not yet establish that the world model is the source of the gain or that total lifecycle cost is lower.

LingBot exposes a budget-fidelity problem. Its local recorded cost is similar to ABot 0.8B, but Clean success is much lower. Official weights perform substantially better on the same local evaluator. This combination favors investigating the adaptation recipe and exposure before scaling the architecture. Forecasts of 80–90% after continuation in source notes are hypotheses and are not used as measured capability or as a basis for cost-to-target claims.

LiLa tests a different service contract. Its compact fixed-task solution is attractive when tasks can be selected through precomputed visual conditions. Current Clean success supports a larger-sample follow-up, not a language-conditioned ranking or a verified consumer-GPU deployment claim. The local stage-2 rollout now provides 222/250 (88.80%) Clean successes, compared with 224/250 (89.60%) for official weights. The next decisive evidence is larger matched sampling, Randomized coverage, a complete DINO-plus-action inference bundle, and direct memory/latency measurements. For applications requiring arbitrary language instructions, the present interface does not satisfy the capability gate regardless of its numerical Clean rate.

Deployment engineering can create value independently of policy learning. PX demonstrates about 2× end-to-end throughput improvement for similar same-concurrency workloads without retraining. The result is actionable for simulation-scale evaluation. Its seed/instruction differences and mean inference statistics do not meet the serving p95 protocol or establish unchanged accuracy on identical episodes. An engineering comparison should therefore report workload integrity, end-to-end time, device-hours, actual steps and policy latency separately.

The highest-value next measurements are those that narrow a decision-relevant gap: serving traces for all policies, Randomized coverage for LingBot and LiLa, and higher-coverage LiLa outcomes. For mecha-

nism research, repeat controlled ablations across seeds. For operational selection, first enforce language/-task interfaces, memory and deadlines, then compare feasible profiles under declared budget weights. This turns the reproduction matrix into a set of falsifiable decisions rather than a catalog of method names.

5.5 PX Evaluation Harness: Systems Reproduction

5.5.1 What Is Being Compared

The fourth source is a deployment/evaluation study of **the existing ABot/PX-M0 Qwen3-VL-2B policy**, not a fourth new action algorithm. It uses three-camera input, 14 absolute joint channels, 50-action chunks, BF16, eager attention and ten DDIM steps. These settings differ from the original ABot action-integration configuration. The October 28 refresh adds complete 50-task systems workloads alongside the original subsets. Historical 100-episode capability scores remain distinct from these ten-episode systems tests.

The original entry launches one model per task process. The harness uses one model service per GPU, simulation containers that share an episode queue, reusable expert-validated seeds and reduced planner initialization for joint control. The implementation can batch requests, but cache access and actual model inference are distinguished. This changes the resource-allocation strategy and makes environment/model dependencies easier to maintain independently.

5.5.2 Experimental Setup

Table 27. PX harness systems-study settings. These experiments benchmark an existing VLA policy rather than training another action model.

Setting	Observed configuration
Policy / accelerator	ABot/PX-M0 Qwen3-VL-2B / H200 NVL
Inputs / action interface	Three cameras / 14 absolute joint channels
Chunk / numerical path	50 actions / BF16, eager attention, ten DDIM steps
Single-GPU timed workload	12 Randomized tasks × ten episodes
Original / shared model instances	One per task process / one per GPU
Primary concurrency / batch cap	Six- and eight-worker comparisons / one
Startup / rollout videos	Included in end-to-end timing / videos disabled
Cross-framework inputs	Accepted seeds and instruction RNG differ; actual steps reported

5.5.3 Measured Speed, Bottlenecks, and Limits

Table 28. Single-GPU systems comparison on 12 Randomized tasks, ten episodes each. End-to-end time includes model startup. Historical PX time is reconstructed from task-directory and log timestamps. Seed acceptance and instruction sampling differ across frameworks, so this is an approximate workload comparison.

Entry	Workers	Successes	Executed steps	End-to-end seconds
Original PX	6	98/120	39,692	2418.949
Harness W6/B1	6	101/120	39,352	1182.580
Harness W8/B1	8	98/120	41,175	1100.330

At equal six-worker concurrency, end-to-end speedup is $2418.949/1182.580 = 2.045\times$; executed-step throughput improves from 16.41 to 33.28 steps/s ($2.028\times$). W8 supplies a further $1.075\times$ wall-time speedup. These gains support the combined system design; they do not isolate model sharing, initialization, queue scheduling or seed reuse individually. No VRAM trace was collected, so a measured peak-memory saving cannot be claimed.

In a separate same-subset scaling run, two GPUs with eight workers each complete 120 episodes in 619.499 seconds, versus 1002.641 seconds on one GPU. Wall time improves by $1.62\times$, but allocation rises from 0.279 to 0.344 GPU-hours (about 23.6% more). More GPUs can minimize completion time while increasing device cost. The throughput of 11.62 episodes/minute is specific to that subset and cannot be extrapolated directly to all 50 tasks.

Table 29. Harness GPU-allocation experiments on the separate 120-episode subset. Evaluation-only times exclude startup and must not be mixed with Table 28's end-to-end times.

Deployment	GPUs	Wall seconds	GPU-hours	Episodes/min
Co-located W8/B1	1	1002.641	0.279	7.18
Co-located W8/B1 per GPU	2	619.499	0.344	11.62
Split model/simulation W16/B1	2	814.538	0.453	8.84

Profiling explains why increasing batch limits does not reliably help. In the split-service B4 retry, 97.87% of 7,969 action requests read cached actions; 170 genuine inference requests produce 147 model calls, mean batch 1.156, with 89.1% of calls still at batch one. Around 95% of cumulative episode time is reset, rendering, observation and physics; the remaining 5% is a residual including networking and waiting, not pure model computation. In that workload, even eliminating the entire residual would yield only about $1/0.95 = 1.053\times$ speedup if all other costs remained fixed. This is a workload-specific Amdahl bound, not a universal limit for VLA inference. Co-location increases the residual and contention.

The harness uses prevalidated seed lists and an independent instruction RNG. A source audit found different accepted seed lists for 11 of 12 sampled tasks relative to the historical entry. Thus 98/120 versus 101/120 cannot establish a capability improvement or strict correctness equivalence. The safe conclusion is an approximately twofold systems speedup for similar workloads, with success counts, actual steps, startup scope and protocol differences disclosed. The harness is assessed as an engineering intervention rather than assigned a separate policy capability score.

5.5.4 Full-Task Validation and Updated Throughput Comparison

The October 26 runs extend the comparison to all 50 Randomized tasks with ten episodes each on two GPUs. All three entries in Table 30 complete 500 episodes and exit successfully. Original PX task results match their final ten-episode log counters; harness aggregate task files independently reproduce their complete, issue-free summaries. Startup-inclusive timings come from the launchers, while rollout-only times remain separate.

Table 30. Updated two-GPU systems comparison over all 50 Randomized tasks. W denotes workers per GPU; harness batch cap is one. Both GPUs are allocated throughout each end-to-end interval. Seed acceptance and instruction sampling differ across execution paths.

Entry	Successes	Steps	End-to-end (s)	GPU-hours	Episodes/min
Original PX / W6	411/500	136,145	4185.99	2.326	7.17
Harness / W6	408/500	138,487	2222.52	1.235	13.50
Harness / W16	401/500	145,542	1814.54	1.008	16.53

At matched W6 concurrency, the full-task harness run has $1.88\times$ higher episode throughput and $1.92\times$ higher executed-step throughput than the original PX entry. W16 reaches $2.31\times$ and $2.47\times$, respectively, relative to original PX. The W16 versus W6 end-to-end speedup is $1.225\times$; actual rollout step counts increase by about 5.1%. Rollout-only times are 2177.461 and 1690.594 seconds for W6/W16, excluding the 45.059/123.946-second startup difference. These results support throughput gains over the full tested task set, while protocol and realized-workload differences preclude attributing the success-count differences to the model or interpreting the speed ratios as isolated component effects.

The refreshed seed-fallback test completes 42/50 successes over five tasks in 354.304 rollout seconds. Six episodes use replacement seeds. In the full-task W6 and W16 runs, 32 and 24 episodes use replacements, respectively. Seed manifests therefore remain necessary even when task names and episode counts match. Additional one-GPU 12-task runs complete 99/120 successes at W12 in 973.547 seconds and 95/120 at W16 in 954.563 seconds, both measured without startup. Their small timing difference is an observed workload result, not a demonstrated universal worker optimum.

Two earlier four-GPU full-task runs record 500 rows but explicitly fail completeness checks: W6 reports infrastructure/partial-result issues on 17 tasks, and W16 on 11 tasks. Their success-rate fields are null, so they are retained as failure diagnostics rather than promoted to capability or throughput winners. A separate Qwen3.5-2B harness launch exits with code one after 7.90 seconds and has no completed summary; this does not invalidate that checkpoint's independently completed historical 100-episode evaluation. The accessible-run inventory, source hashes and task-level system records are retained in `data/current_results_audit.json` and the refresh evidence directory.

5.5.5 Four-GPU Topology and Batch-Cap Comparison

After access was restored, the October 28 batch-comparison directory supplied three further complete, issue-free 120-episode evaluations. All aggregate files reproduce the summary counts. The 12 task names, requested/actual seeds and instructions match across the three runs. Centralized deployment reserves one GPU for the policy server and three for simulation, with eight workers per simulation GPU (24 total); co-location uses four policy services and 16 workers on each of four GPUs (64 total). The worker counts therefore differ as well as the topology.

Table 31. Four-GPU topology and batch-cap observations on the matched 120-episode input manifest. B is the configured maximum model batch, not a measured mean batch size. Rollout-only and startup-inclusive times are separated.

Deployment	Workers	B	Successes	Steps	Rollout (s)	End-to-end (s)
Centralized (1+3 GPUs)	24	1	101/120	37,918	405.435	485.22
Centralized (1+3 GPUs)	24	4	95/120	42,856	472.496	529.27
Co-located (4 GPUs)	64	1	99/120	41,997	414.647	471.01

Centralized B1 has the shortest rollout-only interval, while co-located B1 has the shortest startup-inclusive interval. Increasing the centralized batch cap to four does not reduce the observed wall time, but that run also executes about 13.0% more steps than centralized B1. The approximately 16.5% rollout-time increase cannot therefore be attributed entirely to batching overhead. The cap alone does not establish that inference actually formed batches of four. Although the episode inputs match, differing realized success counts and step lengths, single runs, and the changed simulation concurrency limit causal interpretation. These observations support measuring both startup and realized workload when choosing an evaluation topology; they do not establish a capability ranking or a universal optimal batch size.

6 Conclusion

RoboSeed is a continuing effort to train, evaluate, and openly release robot models for different computing budgets, together with verifiable evidence for model selection. This report connects that community goal to local training, checkpoint evaluation, resource measurements and task-level diagnosis.

We propose RoboSeed as a multidimensional evaluation framework for community researchers selecting and studying robot action models. Its five dimensions connect task capability with downstream training affordability, deployment affordability, locally verified research value and reproduction evidence. Fixed anchors, explicit criteria, alternative preference settings and transparent missing-data rules turn heterogeneous experimental records into inspectable decision profiles. The framework helps researchers choose feasible configurations, assess the evidence behind mechanism claims and prioritize measurements that could change a decision.

The VLA and WAM studies instantiate this framework across four policy families and 12 checkpoint units. Within ABot-M0, the tested smaller backbones retain substantial success with lower training cost, while the completed 9B run shows that higher cost does not ensure better performance under a fixed-update recipe. Task-level robustness and continuous-chain evaluation further reveal limitations hidden by an overall success percentage. These empirical studies supply the evidence from which the framework derives budget-specific assessments.

The extension broadens the evidence to four policy families. OpenWAM local adaptation reaches 91.35% across both RoboTwin scenes using a recorded 262.068 downstream H200 GPU-hours. LingBot's 89.72% official-weight Clean result and 57.78% locally adapted result reveal a large recipe gap requiring controlled investigation. LiLa-WAM official/local stage-2 weights reach 89.60%/88.80% Clean success using five episodes per task and visual task conditions. The 0.80-point observed gap does not establish statistical equivalence. PX demonstrates approximately twofold evaluation throughput improvement for similar workloads at equal concurrency, showing the independent value of deployment engineering.

The five-dimensional assessment makes each judgment explicit: raw capability, scoped training cost, deployment measurements, locally established research value, and evidence completeness. Dashes identify missing measurements and incomplete composite scores, while weighted coverage describes available inputs. The evidence supports budget-specific ABot choices and concrete cross-method follow-ups, while absent serving traces, single-seed training and protocol differences prevent a defensible universal winner.

Reproducible formulas, source snapshots and machine-readable scores make those limits reviewable and future updates straightforward.

7 Future Work

Close decision-critical gaps. Measure warmed batch-one policy p50/p95 latency and peak VRAM on a common H200 protocol, recording native action horizons and all required modules. Complete LingBot/LiLa Randomized evaluation and expand matched LiLa sampling beyond five episodes per task. Audit LiLa's stage-2 learning-rate floor before testing a corrected schedule. These measurements complete missing scoring inputs and enable aggregate assessment.

Test mechanisms under aligned budgets. Compare LingBot exposure and batch schedules without promising an extrapolated success rate. For OpenWAM, isolate predictive supervision and world-to-action interaction while fixing the adaptation budget. Harmonize accepted seeds, instruction sampling, action execution and numerical precision before cross-method attribution, and repeat component ablations across at least three training seeds. Reassess the score policy only with a versioned change to its declared preferences.

Expand backbones and scales. Use the now-complete Qwen3.5-2B evaluation to refine budget choices, then add smaller, larger, and additional VLM families. Use the observed 9B result to test changes in action-expert capacity, feature selection, data coverage, and training duration through controlled ablations. Complement fixed-update comparisons with fixed-GPU-hour studies and multiple training seeds to distinguish budget effects from run variation.

Expand datasets and methods. Evaluate more tasks, embodiments, and data scales, and extend replaceable interfaces to additional VLA methods. Preserve checks of dataset semantics, action conventions, and evaluator versions, while reporting adaptation cost and task performance separately.

Diagnose CALVIN composition failures. Audit the six horizontal-pushing task distributions and signed action statistics, retain per-sequence outcomes and representative failure videos, and compare shorter and longer action chunks under the same sequence list. Evaluate intermediate checkpoints and multiple training seeds before estimating gains. Separate ABCD $\rightarrow$ D from held-out-environment settings such as ABC $\rightarrow$ D.

Rapidly evaluate community techniques. Establish a workflow from compatibility checks and short training runs through screening rollouts and complete evaluation. Verify active acceleration kernels when assessing new architectures.

Strengthen process measurements. Standardize memory sampling across allocated, reserved, and device monitoring readings, with explicit units, peak windows, and device capacity. Publish training-stage runtimes, losses, and anomalies. Record policy p50/p95 latency, simulation throughput, success counts, exclusions, and retries under fixed concurrency, alongside representative success and failure videos.

Connect model releases to continuing report updates. We plan to publish locally trained checkpoints and associated experimental results on Hugging Face, linked from the project repository at `https://github.com/PrimitiveX/RoboSeed`. Each model release should include a model card, effective configuration, normalization statistics, dataset and implementation versions, checksums, and the corresponding evaluation records. Released-weight evaluation and local training outcomes will remain distinct, and unevaluated checkpoints will retain their pending status. Verified download entries and immutable revisions will accompany actual publication. Subsequent report versions will document new models, completed measurements and any resulting changes to selection guidance. Real-robot experiments will test whether the observed simulation trade-offs carry over to deployment.

A Experimental Details and Evidence Completeness

A.1 Dependencies, Interfaces, and Evaluation Settings

Recorded RoboTwin training dependencies are Python 3.12.3, PyTorch 2.8.0+cu128, accelerate 1.5.2, DeepSpeed ZeRO-2, and bf16 without CPU offload. Qwen3-VL uses transformers 4.57.0, and Qwen3.5 uses 5.15.1. These are run-specific records; reproduction should pin compatible combinations. Effective configuration snapshots saved during training take precedence over repository defaults modified afterward.

The LIBERO-plus script inherits the four suite-specific step limits and settling period. Reproduction and external comparisons should check these settings, executed action chunks, random states, and environment versions. Complete task coverage alone does not imply equivalence to every public evaluation protocol.

Table 32. Key interfaces and evaluation settings. LIBERO-plus entries describe this script snapshot rather than a universal protocol for other implementations.

Setting	RoboTwin	LIBERO / LIBERO-plus	RoboCasa GR1
Training updates	150k	40k / no extra training	50k
Global batch	64	64	64
State / action dimensions	14 / 14	7 / 14 in config; state input to be verified	58 / 29
Training action horizon	50	10	16
Integration steps	4	4	4
Training seed	42	42	42
Evaluation seed	Launch argument 0	Script default 7	Not recorded in summary
Episodes per task	100 per scene	50 / 1 per variant	50
Environment step limit	Task dependent	220 / 280 / 300 / 520 by suite; 10 settling steps	720
Rollout videos	Disabled	Available	Available

A.2 Available Evidence and Remaining Measurements

Table 33. ABot evidence completeness after the October 28 refresh. Additional method results are audited separately in Appendix D.

Evidence	Current status	Follow-up
RoboTwin success	Original results checked against final log counters; 50 tasks per scene	Multiple training seeds and external protocol checks
Configurations and final weights	Six evaluated RoboTwin configurations and dataset-specific final weights; file sizes measured	Versioned release configurations and weight checksums
Loss convergence	Observations extracted from console logs	Standard exports of raw metrics
Training memory	Approximate recorded mean and maximum readings	Raw traces, units, and capacity checks
Rollout time and concurrency	Workflow times and concurrency limits recorded	Policy latency, task time, and worker activity
LIBERO / RoboCasa counts	Reconstructed video verdicts and task denominators	Independent episode tables and exception logs
LIBERO-plus coverage and duplicates	All variants covered; duplicates and conflicts preserved	Confirm rerun replacement using run IDs and logs
CALVIN chains and task counts	Final aggregate record selected; 34 task types and chain totals checked	Per-sequence traces, evaluator commit, and failure videos
Public repository and weight entries	Project repository provided; release materials under expansion	Weight download links and dependency documentation

A.3 Reproducing the Data and Figures

The delivered `data/` directory contains six completed RoboTwin configurations and task tables, six training-loss series, task outcomes for LIBERO, LIBERO-plus, and RoboCasa, and LIBERO-plus category statistics and duplicate audits. CALVIN adds prefix counts, all 34 conditional task counts, failure-group statistics, final-run loss observations, the original aggregate JSON, and a duplicate-key selection audit. The refreshed `source_manifest.json` records source paths and evidence hashes. Separate extension records contain source manifests, task counts, checkpoint metadata and score profiles; Appendix D gives their audit and regeneration procedure. Original directory aliases appear only in provenance materials; method names in the report remain consistent. ABot video outcomes are reconstructed from environment-success labels in filenames, rather than independently relabeled through frame-by-frame inspection of every rollout.

The read-only `scripts/collect_data.py` extracts evidence and checks coverage and denominators. The

scripts `make_tables.py` and `draw_figures.py` generate manuscript tables and figures from shared data. Figures are exported as PDF, SVG, and PNG; framework diagrams remain editable vectors. All new illustrations use blue, teal, white, and muted gray. Original experiment files and weights are preserved.

B Complete RoboTwin Task Counts

Every numeric column reports successes out of 100 episodes, numerically equal to success rate in percent. C and R denote Clean and Randomized. Q3 denotes Qwen3-VL and Q3.5 denotes Qwen3.5. All six backbone pairs have complete evaluations, including Qwen3.5-2B.

Table 34. Success counts for all 50 RoboTwin tasks.

	Q3.5-0.8B		Q3-2B		Q3-4B		Q3.5-2B		Q3.5-4B		Q3.5-9B	
Task	C	R	C	R	C	R	C	R	C	R	C	R
adjust_bottle	100	100	100	100	100	100	100	99	100	99	100	100
beat_block_hammer	83	82	94	88	94	92	86	83	89	83	96	87
blocks_ranking_rgb	91	93	93	93	92	94	90	84	98	91	94	90
blocks_ranking_size	57	59	67	66	66	65	54	74	74	69	72	66
click_alarmclock	2	5	5	6	10	29	6	15	22	13	3	9
click_bell	0	2	1	7	57	58	20	12	15	28	5	15
dump_bin_bigbin	97	93	97	97	96	97	97	96	97	99	96	99
grab_roller	100	100	100	100	100	100	100	100	100	100	100	99
handover_block	72	58	95	83	86	79	85	85	82	74	88	75
handover_mic	100	99	100	100	98	98	100	100	100	99	98	99
hanging_mug	23	24	27	28	31	35	28	36	30	34	38	36
lift_pot	99	100	100	100	100	100	100	99	100	98	100	100
move_can_pot	83	71	87	83	84	74	82	84	82	86	73	75
move_pillbottle_pad	93	86	93	92	96	95	94	92	93	90	96	94
move_playingcard_away	97	95	97	98	100	98	99	98	99	100	97	99
move_stapler_pad	48	52	48	54	47	49	76	73	72	77	72	60
open_laptop	95	98	98	99	97	99	95	100	97	100	97	98
open_microwave	84	80	88	75	89	86	79	81	73	79	82	64
pick_diverse_bottles	41	59	69	67	71	59	65	69	63	61	64	57
pick_dual_bottles	72	75	74	69	87	70	68	72	85	83	72	67
place_a2b_left	81	90	88	87	93	96	92	93	95	97	93	94
place_a2b_right	78	83	86	83	91	92	90	91	91	88	95	94
place_bread_basket	85	81	85	81	87	77	86	85	92	86	91	86
place_bread_skillet	89	81	84	83	82	89	89	77	94	86	91	88
place_burger_fries	99	98	96	98	95	98	100	100	99	95	99	98
place_can_basket	70	63	75	76	79	75	73	77	81	72	70	72
place_cans_plasticbox	100	92	100	96	100	99	100	97	98	96	99	98
place_container_plate	100	99	99	100	98	99	100	100	100	100	99	100
place_dual_shoes	85	82	80	79	82	84	91	83	84	85	75	81
place_empty_cup	99	100	100	100	100	100	99	99	100	100	99	100
place_fan	94	91	96	98	99	97	93	92	94	97	98	94
place_mouse_pad	68	68	75	78	73	70	77	77	82	73	76	73
place_object_basket	88	88	92	89	88	86	97	86	89	88	90	86
place_object_scale	88	80	93	93	95	90	96	86	94	93	92	94
place_object_stand	96	88	90	91	94	86	94	91	97	92	97	92
place_phone_stand	77	93	87	98	94	97	91	95	91	96	91	99
place_shoe	94	96	96	97	95	100	91	100	94	97	93	99
press_stapler	95	94	98	97	93	94	95	93	96	94	87	88
put_bottles_dustbin	67	79	79	75	75	80	82	74	79	89	74	81
put_object_cabinet	91	91	96	96	87	90	92	86	91	92	90	95
rotate_qrcode	90	90	90	93	91	94	84	88	91	94	84	91
scan_object	74	83	78	88	89	87	83	81	81	81	79	77
shake_bottle	100	100	100	100	100	100	100	98	98	100	100	100
shake_bottle_horizontally	100	99	100	100	100	99	99	99	100	99	100	100
stack_blocks_three	82	75	93	84	91	82	84	81	84	80	87	87
stack_blocks_two	100	97	98	99	96	99	95	96	98	93	100	98
stack_bowls_three	89	80	79	77	86	83	85	78	82	80	85	88
stack_bowls_two	98	97	96	98	96	98	98	96	99	98	99	97
stamp_seal	64	65	68	86	75	86	68	78	78	83	74	78
turn_switch	60	71	53	70	60	63	67	68	71	69	49	67

C CALVIN Configuration, Training, and Complete Task Counts

The saved configuration requests a minimum learning rate of 10^{-6}, whereas the last logged scalar learning rate is 4×10^{-7}. Scheduler and parameter-group conventions must be reconciled before claiming an exact effective rate. The source notes describe acceleration dependencies for this later training environment; a matched kernel ablation has not been performed. These details accompany the result without implying a measured speedup relative to earlier 4B runs.

Every row below uses the actual number of reached attempts. A task later in a sequence is observed only

Table 35. CALVIN reproduction details from the saved configuration and inspected evaluation interfaces.

Setting	Recorded value
Training / evaluation environments	ABCD / D
Demonstrations / frames	17,870 / 1,071,743; converted LeRobot v2.1
Updates / batch / seed	34,000 / 64 / 42
Frozen encoder	VGGT spatial module
Action expert / integration steps	DiT-B, 16 layers / configured 4
Training / executed action horizon	8 / 8 in the inspected shared client
State / action layout	Configured 16 / 14; active CALVIN action channels: 7
Policy observations	Static and wrist cameras plus language; no explicit robot-state field
Normalization	Saved statistics, min–max action unnormalization, right-arm channel selection
Subtask step limit / stop rule	360 / terminate chain at first failure
Evaluation clients / videos	One simulation client / debug video output disabled by default
Training / recorded evaluation time	14:39:51 / approximately 10:16:00

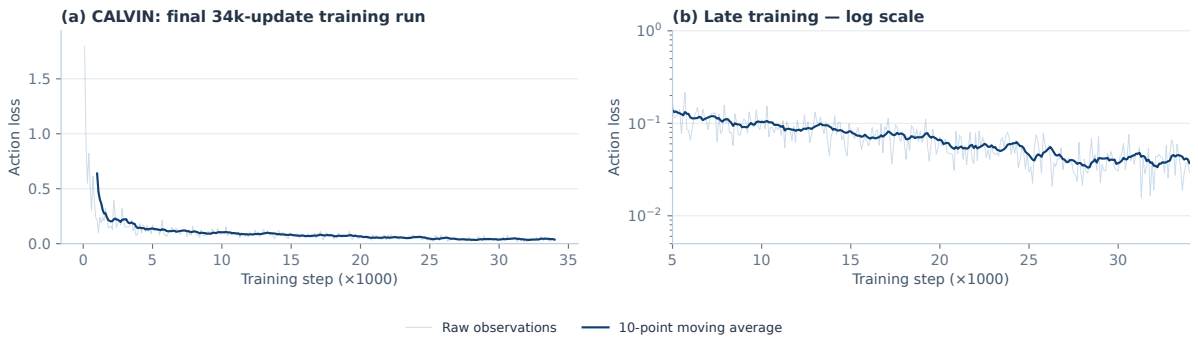


Figure 17. Action loss from the final CALVIN training run. All 340 observations at 100-update intervals are retained; the moving average uses ten observations. Nine earlier observations from preliminary runs in the same console file are excluded from this curve and accounted for in the audit. Final action loss is 0.02949.

when its predecessors succeed. The table includes `close_drawer` exactly once and contains 34 unique task types. Totals are 3,625 successes and 4,054 attempts; 26 task types achieve at least 85% conditional success.

Table 36. Complete CALVIN conditional task counts for the selected final evaluation.

Task	Success / attempts	Rate (%)
close_drawer	182/182	100.00
lift_blue_block_drawer	19/19	100.00
lift_blue_block_slider	100/123	81.30
lift_blue_block_table	143/143	100.00
lift_pink_block_drawer	11/11	100.00
lift_pink_block_slider	109/128	85.16
lift_pink_block_table	120/129	93.02
lift_red_block_drawer	17/17	100.00
lift_red_block_slider	106/122	86.89
lift_red_block_table	153/155	98.71
move_slider_left	231/233	99.14
move_slider_right	252/252	100.00
open_drawer	307/320	95.94
place_in_drawer	147/151	97.35
place_in_slider	258/298	86.58

Task	Success / attempts	Rate (%)
push_blue_block_left	50/70	71.43
push_blue_block_right	23/70	32.86
push_into_drawer	88/102	86.27
push_pink_block_left	33/73	45.21
push_pink_block_right	21/64	32.81
push_red_block_left	50/74	67.57
push_red_block_right	19/72	26.39
rotate_blue_block_left	66/66	100.00
rotate_blue_block_right	70/77	90.91
rotate_pink_block_left	54/55	98.18
rotate_pink_block_right	45/68	66.18
rotate_red_block_left	60/61	98.36
rotate_red_block_right	69/72	95.83
stack_block	153/165	92.73
turn_off_led	138/150	92.00
turn_off_lightbulb	134/135	99.26
turn_on_led	168/168	100.00
turn_on_lightbulb	164/164	100.00
unstack_block	65/65	100.00

D New Reproduction Counts and Provenance

The October 28 refresh rechecks the supplied experiment roots against the previous report. It adds the completed ABot Qwen3.5-2B and local LiLa-WAM outcomes, updates their resource/evidence records, and incorporates the latest accessible PX systems runs. Existing ABot backbone, LIBERO, LIBERO-plus, RoboCasa and CALVIN observations remain unchanged where no new outcome is found. The extension manifest now lists 235 source records, with byte counts and SHA-256 hashes; a separate refresh audit inventories new, incomplete and inaccessible candidates. Large weights are neither loaded nor copied nor hashed: artifact sizes, saved states, configurations and resolved checkpoint paths identify the studied weights, without claiming a cryptographic tensor audit.

`scripts/collect_data.py` now validates all six ABot RoboTwin configurations against their final 100-episode task counters. `scripts/collect_extension.py` imports those 600 task/scene rows and adds 400 LingBot/OpenWAM/LiLa rows, yielding 1,000 unique checkpoint/scene/task records. LiLa official and local worker JSONs are independently checked against their final log counters and corresponding checkpoint settings by `scripts/audit_current_results.py`. The official backup directory is kept separate from the reused current result directory. OpenWAM's existing rate records retain the documented 100-episode denominator; no independent episode-verdict reconstruction is claimed for them.

The refresh audit also checks every accessible harness summary and retains unsuccessful runs. Two new full-task harness runs reproduce all 500 episode outcomes from 50 aggregate task files each; the original PX full-task comparator reproduces 411/500 successes and 136,145 executed steps from task logs. Startup-inclusive launcher intervals remain distinct from rollout-only summary times. The Wan2.1-VACE-1.3B saved 50k state has no rollout results and remains outside the evaluated checkpoint profiles. Access to the previously restricted October 28 four-GPU batch-comparison directory was restored. All three 120-episode runs are now checked against their task aggregates and included in Section 5.5; their task/seed/instruction manifests match. Exact scanned paths, access limits, configurations and evidence hashes are retained in `data/current_results_audit.json`.

The offline `scripts/assessment.py` regenerates `data/assessment.json`, the score and sensitivity rows, the new result/task tables and Figure 16. Its inputs are the saved unit metadata in `data/extension_units.json` and task counts in `data/extension_tasks.csv`, with policy version 1.1. The JSON preserves per-scene Wilson intervals, all value/evidence flags, raw costs, scoring anchors, preference weights, scalar scores and input coverage. Missing measurements and incomplete composites are `null` in JSON and dashes in tables. Version 1.1 replaces the earlier missing-input ranges with this explicit missing-value policy; observed measurements and scoring anchors are unchanged. New measurements under the same rules should update the evidence snapshot. Source paths and hashes are in `data/extension_manifest.json`, and copied records are under `data/extension_sources/`.

D.1 Per-Method Experimental Results and Training Curves

Chapters 3 and 4 place experimental results immediately after each method description. The additional training observations are generated by `scripts/method_experiments.py` with source access us-

ing `--collect`, or regenerated offline from the saved CSVs without that flag. The separate `data/method_experiments_manifest.json` records six source hashes, extraction decisions, native first/last losses, and LiLa epoch timers. The original checkpoint outcomes and scoring rules remain unchanged by this extraction.

LingBot contributes 50,000 update records; OpenWAM contributes 80,000 selected progress-counter records with four-decimal display precision and a possible one-update offset. LiLa contributes 23,643 and 7,881 sampled-update records for completed stage-1 and stage-2 prefixes. Its seven/four partial-next-epoch observations are excluded explicitly. All retained observations remain in the CSVs even when light figure traces are subsampled; moving averages use the full series. Model-native loss components are plotted separately and are not averaged across methods. The logging implementations used to interpret OpenWAM counters and LiLa loss components are copied into the method-experiment source directory.

D.2 All 50 New Task Counts

L-O and L-L denote official and local LingBot Clean successes out of 100. W-O and W-L give official and local OpenWAM Clean/Randomized successes, each out of 100. Li-O and Li-L give official and local LiLa Clean counts out of five. LiLa and LingBot Randomized remain unavailable in this evidence set. Sections 3.3, 5.5, 4.1 and 4.2 describe the different protocols; row alignment does not make episodes paired.

Table 37. Complete task counts for the new model reproductions.

Task	L-O	L-L	W-O (C/R)	W-L (C/R)	Li-O	Li-L
adjust_bottle	100	99	100/99	96/99	5/5	5/5
beat_block_hammer	91	45	98/99	100/97	4/5	5/5
blocks_ranking_rgb	92	58	100/96	98/100	5/5	5/5
blocks_ranking_size	76	36	78/77	76/83	4/5	5/5
click_alarmclock	64	62	97/100	96/99	5/5	5/5
click_bell	58	18	100/100	100/100	5/5	4/5
dump_bin_bigbin	96	90	97/97	96/97	4/5	5/5
grab_roller	100	100	100/99	100/99	5/5	5/5
handover_block	95	5	96/97	98/89	4/5	5/5
handover_mic	93	91	89/95	94/96	5/5	5/5
hanging_mug	48	44	43/54	36/34	3/5	4/5
lift_pot	100	16	100/100	100/99	5/5	4/5
move_can_pot	88	16	99/99	96/98	5/5	5/5
move_pillbottle_pad	95	57	99/99	100/96	5/5	5/5
move_playingcard_away	100	91	100/100	100/99	5/5	5/5
move_stapler_pad	70	33	98/98	83/91	3/5	2/5
open_laptop	98	97	99/99	96/100	5/5	5/5
open_microwave	84	62	96/73	73/71	4/5	2/5
pick_diverse_bottles	85	14	83/84	81/88	3/5	4/5
pick_dual_bottles	89	10	95/98	94/88	5/5	5/5
place_a2b_left	93	82	97/98	98/95	3/5	1/5
place_a2b_right	94	91	100/96	98/92	5/5	4/5
place_bread_basket	93	62	94/97	96/90	4/5	4/5
place_bread_skillet	94	59	90/88	88/89	4/5	4/5
place_burger_fries	96	73	97/99	99/96	5/5	5/5
place_can_basket	63	38	78/76	81/79	3/5	4/5
place_cans_plasticbox	89	2	99/100	98/95	5/5	5/5
place_container_plate	96	95	100/97	100/99	5/5	5/5
place_dual_shoes	93	49	96/91	93/87	4/5	1/5
place_empty_cup	100	61	100/100	100/100	5/5	4/5
place_fan	84	58	97/95	99/100	5/5	5/5
place_mouse_pad	97	51	99/99	97/98	4/5	3/5
place_object_basket	92	56	93/89	90/86	3/5	5/5
place_object_scale	95	78	95/99	88/93	5/5	5/5
place_object_stand	96	69	97/97	99/93	5/5	5/5
place_phone_stand	90	76	93/95	94/99	5/5	5/5
place_shoe	99	84	100/100	99/98	5/5	5/5
press_stapler	100	93	90/94	79/91	5/5	5/5
put_bottles_dustbin	83	13	94/95	89/91	5/5	5/5
put_object_cabinet	96	62	88/92	85/75	5/5	5/5
rotate_qrcode	94	73	93/96	91/89	4/5	4/5
scan_object	96	16	97/94	95/89	5/5	5/5
shake_bottle	100	96	100/100	99/100	5/5	5/5
shake_bottle_horizontally	100	99	100/100	99/100	5/5	5/5
stack_blocks_three	97	36	97/96	91/93	5/5	5/5

Continued on next page

Task	L-O	L-L	W-O (C/R)	W-L (C/R)	Li-O	Li-L
stack_blocks_two	100	46	100/100	100/100	5/5	5/5
stack_bowls_three	78	45	85/80	80/82	5/5	4/5
stack_bowls_two	96	82	97/94	95/96	5/5	5/5
stamp_seal	78	37	96/98	87/95	2/5	5/5
turn_switch	82	63	72/65	45/57	4/5	4/5

References

Wei Wu et al. From Foundation to Application: Improving VLA Models in Practice. 2026. https://arxiv.org/abs/2607.06403.

Yuran Wang et al. OpenWAM: An Open, Modular Exploration Towards Systematic World-Action Model Pretraining. 2026. https://arxiv.org/abs/2609.07398.

Fan Yang et al. LiLa-WAM: Lightweight Latent Reasoning World-Action Model for Robotic Manipulation. 2026. https://arxiv.org/abs/2608.03701.

Yandan Yang et al. ABot-M0: VLA Foundation Model for Robotic Manipulation with Action Manifold Learning. 2026. https://arxiv.org/abs/2602.11236.

Moo Jin Kim et al. OpenVLA: An Open-Source Vision-Language-Action Model. 2024. https://arxiv.org/abs/2406.09246.

Mustafa Shukor et al. SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics. 2025. https://arxiv.org/abs/2506.01844.

Tianxing Chen et al. RoboTwin 2.0: A Scalable Data Generator and Benchmark with Strong Domain Randomization for Robust Bimanual Robotic Manipulation. 2025. https://arxiv.org/abs/2506.18088.

Bo Liu et al. LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning. 2023. https://arxiv.org/abs/2306.03310.

LIBERO-plus. Official implementation: In-depth Robustness Analysis for Vision-Language-Action Models. https://github.com/sylvestf/LIBERO-plus.

RoboCasa. GR1 Tabletop Tasks for GR00T N1. https://github.com/robocasa/robocasa-gr1-tabletop-tasks.

Oier Mees, Lukas Hermann, Erick Rosete-Beas, and Wolfram Burgard. CALVIN: A Benchmark for Language-Conditioned Policy Learning for Long-Horizon Robot Manipulation Tasks. 2022. https://arxiv.org/abs/2112.03227. Official implementation: https://github.com/mees/calvin.